The Benchmark Chooses the Winner:

Measuring, Tuning, and Composing Small Safety Guards
Across Objectives and High-Compliance Business Domains

Reza Rahimi
JazzX AI, Los Altos, CA, USA
reza.rahimi@jazzx.ai

July 16, 2026

Abstract

The problem. A safety *guard* is a small model that labels each incoming request `safe` or `unsafe` before an assistant acts on it. Teams build one by fine-tuning a base model and keeping whichever variant scores best on a public benchmark — then read that score as "how good the guard is." It is not: a guard's score is *co-produced* by the benchmark it is read on, the training objective, the decision threshold, and the domain, so the leaderboard-best guard can be the wrong guard for the traffic you actually serve.

Why it matters, and what it costs, in high-compliance domains. In finance, health, law, and mortgage lending the dangerous requests often read as *polite, compliant text* — e.g. a mortgage inquiry that quietly steers by a protected class. A guard tuned to look strong on generic benchmarks can silently miss these domain violations, and can treat protected groups differently in ways accuracy never surfaces — turning a good-looking score into regulatory, fair-lending, and reputational exposure. The costs are measurable: fine-tuning a guard to raise its familiar-data score, then deploying at a calibrated threshold, more than *triples* false alarms on unseen traffic ($8.1\% \to 15.5\%$) while *dropping* hard-attack recall ($78.0\% \to 60.0\%$); a cutoff that looked safe in calibration decays under sub-standard-deviation drift; and at realistic ($\sim 1\%$) prevalence, precision collapses and the very *ranking* of guards reorders.

What we propose. A disciplined, fully reproducible way to measure what a guard's fine-tune actually bought — and to choose guards for regulated products. On one fixed panel of four compact checkpoints (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B) we compare each guard only against its *own* pre-tuning base on identical data (so a change is attributable to the fine-tune, not to a model swap); separate *familiar* (represented) from *unseen* (transfer) test data; extend the comparison across training objectives (SFT/DPO/GRPO, pre-registered); add a *retraining-free* composition that averages the base and its fine-tune; and evaluate in depth on an HMDA-grounded, *dual-labeled* mortgage benchmark with a protected-class fairness gate, plus an external expert-labeled finance/health/law set (ExpGuard, 2,275 rows).

What we find. Fine-tuning *specializes* but does not transfer: it lifts familiar-data ranking to a near-common ceiling (≈ 0.98; $+0.3234$ on average) yet changes held-out transfer by only -0.0589 on average — an *attractor* that pulls every model to a benchmark-fixed endpoint, so "stronger bases specialize more" is arithmetic, not skill. Composition recovers much of the lost transfer for free ($+0.076$ over the fine-tune). And a general-safety score does not cover domain compliance: general guards rank mortgage violations only moderately and their fairness varies widely, and the best finance/health/law guard (SmolLM3-3B, AP 0.88–0.96) is *not* the largest model and *not* the guard the mortgage benchmark prefers — the winner flips with the benchmark.

What is new. Prior work either compares *different* models on a leaderboard or shows generically that fine-tuning can erode safety; both conflate the fine-tune's effect with differences

between models. Our paired same-checkpoint estimand isolates it, and we contribute the attractor/endpoint-invariance reframing, a retraining-free transfer-recovery operator, and a dual-labeled domain benchmark with a fairness-invariance gate — kept honest by never pooling weaker and stronger evidence tiers and by regenerating every number from committed per-row scores.

Impact for high-compliance business. Do not choose a guard by a single leaderboard number: measure it against its own base, on your traffic, at your prevalence; you can recover cross-domain robustness *without* paying to retrain; and you must gate on domain-grounded compliance *and* fairness, which general-safety accuracy does not capture. All evidence here is *retrospective and estimation-only* on a fixed panel (mortgage labels are LLM-judge, not expert-adjudicated), so no causal or fair-lending claim is licensed; code, data manifests, and the frozen benchmark are public (Section 12; `https://github.com/rrahimi-uci/guard-ranking-fragility`).

Contents

1 Introduction: why "is this guard good?" is the wrong question

In one paragraph

A "guard" is a small model that reads an incoming request and labels it `safe` or `unsafe` before an assistant acts on it. The common recipe is: take a general chat model, fine-tune it into a guard, and report its benchmark score — usually compared against *other* models. That hides the quantity a practitioner actually needs: what did the fine-tune change relative to the *same model before tuning*, and does that change survive on data the guard never trained on? This report answers that at three scales (does tuning transfer? does the *objective* matter? can we compose to recover?) and then asks whether any of it holds where it matters most — *regulated domains* where a violation can read as perfectly polite text.

Why an engineering leader should care. The gap between a benchmark score and field behavior is not academic. A guard chosen on a leaderboard — or "improved" by a quick fine-tune — can quietly do three expensive things at once in production: **(1)** raise false alarms that block legitimate users (we measure pooled transfer false alarms climbing $8.1\% \rightarrow 15.5\%$), **(2)** miss the hardest attacks it was meant to stop (HarmBench recall $78.0\% \rightarrow 60.0\%$), and **(3)** — in a regulated domain — pass a policy violation that reads as polite text, or treat a protected group differently, which is a compliance and fair-lending problem, not merely a lower number. This report is both a way to catch those failures *before* deployment and a set of concrete choices — when to fine-tune, when to instead *compose*, and when a general guard is simply not enough for a regulated domain (the decision guide, Section 10) — written so a practitioner can act on them.

Converting a general instruction model into a safety guard with a short parameter-efficient fine-tune [13] and reporting its score on a public suite [4] is now standard. A post-tuning leaderboard number, however, conflates two different things: raising the score on benchmark *sources represented in training* versus improving *transfer* to datasets the guard never saw. This report is organized around one thesis — *the benchmark co-produces the verdict* — and four questions on a single shared panel of checkpoints:

1. **Does fine-tuning transfer?** (Section 5) — or does the guard specialize to its training sources?
2. **Does the objective matter?** (Section 6) — SFT vs DPO vs GRPO. PENDING the locked run.
3. **Can we recover transfer without retraining?** (Section 7) — by composing base + adapter.
4. **Does any of it survive a change of domain?** (Section 8) — across law, finance, health, and mortgage.

What is new here. Prior work shows fine-tuning can weaken safety behavior [22], that guards overfit their training policy [32], and that a plain instruction model can rival a purpose-built guard [4] — but those compare *different* models, which confounds "the fine-tune changed the guard" with "the two models differ." Our through-line removes that confound with a *paired, same-checkpoint* measurement and then pushes it across objectives, a composition remedy, and four domains. Concretely, the novel contributions are:

1. **A paired estimand** that cleanly separates represented-source gain from held-out transfer, with a fixed-panel hierarchical bootstrap (Section 5).
2. **The "attractor" finding:** SFT drives every checkpoint to a benchmark-fixed endpoint, so

the popular reading "stronger bases specialize more" is arithmetic; the sharper, falsifiable claim is *endpoint invariance* (Section 5.6).

3. **A pre-registered objective axis** (SFT/DPO/GRPO) with a single-token argument for why GRPO should end KL-close (Section 6), committed before the run as an anti-HARKing timestamp.
4. **A retraining-free composition operator** that recovers transfer as an ensemble diversity gain, with the mechanism (base-correlated adapter errors) made explicit (Section 7).
5. **A dual-labeled, HMDA-grounded mortgage benchmark** ($G \times D$) with a protected-class fairness-invariance gate, plus an external, expert-annotated finance/health/law replication (Section 8).
6. **A reproduce-from-committed-scores pipeline:** one entry point regenerates every table and figure and asserts byte-identity against the paper (Section 12).

*What is **not** new:* we introduce no new model, metric, or training algorithm, and make no causal, universal, or deployment claim — the output is a reproducible, estimation-only characterization of this fixed panel plus one external domain replication.

Scope. English input-prompt classification; binary `safe`/`unsafe`; four compact checkpoints (1.5–4B); one LoRA recipe; objectives SFT/DPO/GRPO; four regulated domains. Not a leaderboard, not a fair-lending audit, not a deployment recommendation.

2 Background you'll need

This report is written to be read by someone with a first course in statistics, a working idea of what fine-tuning a small language model means, and a lay familiarity with mortgage lending. Everything technical the four acts rely on is defined once here, from the ground up, with the exact equations and a worked example wherever a number would otherwise be mysterious. Nothing in this section reports a result; it is the vocabulary.

The one-paragraph orientation

A *guard* is a small model placed in front of an assistant: it reads an incoming request and labels it `safe` or `unsafe` before anything acts on it. We build each guard by *fine-tuning* a general chat model, and we score it by how well its numeric output *ranks* truly-unsafe requests above safe ones. The central move of this report is to always compare a fine-tuned guard against the *same model before tuning*, on identical inputs, and to separate two kinds of test set: benchmarks whose data the guard *trained on* (represented) and benchmarks it *never saw* (transfer). Almost every subtlety below is a way of making that comparison honest; Figure 1 previews the whole design on one page.

2.1 The guard: a single-token logit-difference head

We do not ask the guard to write a paragraph explaining its decision. We ask it for exactly one word and look only at how strongly it leans toward that word. Concretely, a prompt x is wrapped in a fixed instruction template asking for a one-word verdict, and we read the model's raw output scores — its *logits* — at the final position for the two verdict words `safe` and `unsafe`. A logit is the model's un-normalized, pre-probability score for a candidate next token: larger means the model favors that token. Writing $z_{\text{unsafe}}(x,t)$ and $z_{\text{safe}}(x,t)$ for those two logits at the final prompt

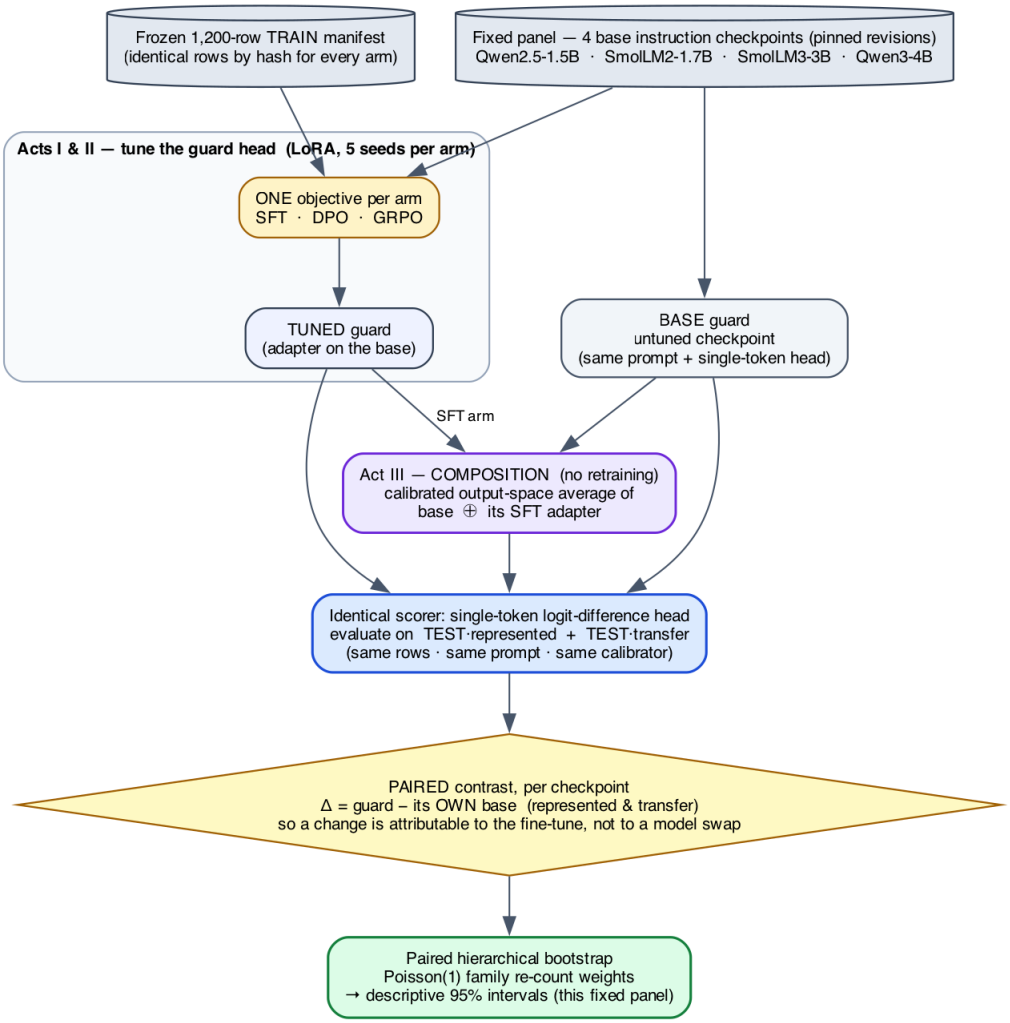

Figure 1: **The study at a glance (Acts I–III).** From one fixed panel and one frozen training manifest, every guard we compare — the untuned **base**, each **tuned** arm (SFT/DPO/GRPO), and the retraining-free **composition** — is read by the *same* single-token scorer on the *same* represented and transfer test rows, and each is compared only against its *own* base. That paired difference Δ, aggregated by a family-aware bootstrap, is the estimand. Act IV then adds the mortgage and finance/health/law domain evaluations on top of this same machinery.

position t, the guard's stored score is their difference,

$$s(x) \;=\; z_{\text{unsafe}}(x,t) \;-\; z_{\text{safe}}(x,t), \tag{1}$$

which is positive when the model leans `unsafe`, negative when it leans `safe`, and near zero when it is undecided. To read $s(x)$ as a probability we pass the same two logits through a two-way softmax,

$$p(\text{unsafe} \mid x) \;=\; \frac{\exp\big(z_{\text{unsafe}}\big)}{\exp\big(z_{\text{safe}}\big) + \exp\big(z_{\text{unsafe}}\big)}. \tag{2}$$

For example, if $z_{\text{unsafe}} = 2.0$ and $z_{\text{safe}} = 1.0$, then $s(x) = 1.0$ and $p(\text{unsafe} \mid x) = e^2/(e^2+e^1) \approx 0.73$. This is what we mean by a "single-token logit-difference head": one forward pass, two numbers, one score. The raw logits are the canonical stored value; probabilities in Equation (2) and any temperature-rescaled version are *derived* from them. Reading the model's own generated verdict text is deliberately never used as the primary score, because a free-text answer is harder to score consistently and hides the model's actual confidence.

2.2 Base, SFT, and LoRA

The *base* is the general instruction-tuned chat model *before* we turn it into a guard. It is not untrained — it can already follow the "answer `safe` or `unsafe`" instruction zero-shot — so "base" throughout means "before the guard fine-tune," not "blank." *Supervised fine-tuning* (SFT) then trains that model on labeled examples: for each training prompt we know the correct verdict, and we nudge the model to raise the probability it assigns to that verdict word. Rather than update all of the model's weights, we use *LoRA* (Low-Rank Adaptation) [23]: we freeze the original weights and insert a small number of extra trainable parameters (a low-rank "adapter") into selected weight matrices. LoRA is cheap, fast, and reversible — you can ship the tiny adapter on top of the frozen base — and it is the standard way guards are built in practice [13]. The precise adapter size and which matrices it touches are part of the frozen recipe in Section 3.

2.3 Average precision, macro-AP, and the accuracy trap

Because $s(x)$ is a continuous score, we can grade a guard *without* committing to a cutoff, by asking how well it *ranks*. *Average precision* (AP) collapses the whole precision–recall curve into a single number in $[0, 1]$: it is near 1 when the guard tends to give genuinely unsafe prompts higher scores than safe ones, and it needs no threshold. Formally, sweeping the cutoff from high to low, AP is the recall-weighted average of the precision achieved at each true-positive, $\text{AP} = \sum_n (R_n - R_{n-1})\, P_n$, with P_n, R_n the precision and recall after the n-th ranked item; we use the tie-aware, non-interpolated `scikit-learn` definition so the number is exactly reproducible.

A worked AP example

Rank five prompts by their guard score, highest first, and write their true labels: [unsafe, unsafe, safe, unsafe, safe]. There are three unsafe prompts, so each one contributes recall $\frac{1}{3}$. The precision at each unsafe prompt as we go down the list is $\frac{1}{1}$=1.00 (rank 1), $\frac{2}{2}$=1.00 (rank 2), and $\frac{3}{4}$=0.75 (rank 4). So $\text{AP} = \frac{1}{3}(1.00) + \frac{1}{3}(1.00) + \frac{1}{3}(0.75) \approx 0.917$. A perfect ranking (all three unsafe prompts on top) would score 1.00; ranking the safe prompts first would score much lower.

Macro-AP computes AP separately on each benchmark and then averages those per-benchmark values with *equal weight*, so one large benchmark cannot dominate a handful of small ones; this is the primary metric in every act.

Why not just report accuracy?

On a test set that is 95% safe, a guard that blindly answers `safe` to everything scores 95% accuracy while catching *zero* attacks. Accuracy rewards guessing the majority class; AP and macro-AP instead reward putting the unsafe prompts on top, which is the behavior a guard is for. This is why ranking, not accuracy, is the currency of this report.

2.4 Two evaluation regimes: represented-source vs. dataset-held-out transfer

The single most important distinction in this report is *where the test data came from*. **Represented-source** test rows are held-back examples drawn from a dataset the guard *trained on* (different rows, same source). **Dataset-held-out transfer** rows come from datasets whose examples were *never used in training at all*. A gain that appears on represented-source tests but not on transfer tests is precisely the signature of a guard *specializing* to its training sources rather than becoming a broadly better moderator. One honesty caveat travels with the word "held out": it means *dataset-held-out* (those rows were withheld from fine-tuning), *not* sealed away from the researchers during development — a distinction that forces the retrospective framing discussed in Section 2.6.

2.5 From scores to decisions: calibration, operating point, TPR/FPR, macro vs. pooled

Ranking is threshold-free, but a deployed guard must eventually say `safe` or `unsafe`. An *operating point* is the single cutoff on $s(x)$ that converts scores into hard decisions. Picking it trades two error rates against each other: the *true-positive rate* (TPR, or recall) is the fraction of genuinely unsafe prompts the guard flags, and the *false-positive rate* (FPR) is the fraction of genuinely safe prompts it wrongly flags. Raising the cutoff lowers both; lowering it raises both. Before thresholding we *calibrate*: we fit a single positive *temperature* that rescales the logits (temperature scaling [18]) so the derived probabilities in Equation (2) are better behaved. Crucially, both the temperature and the cutoff are fit only on a separate held-back *calibration* split — never on the transfer or stress data we report on — so the threshold is not quietly tuned to the test.

Two error rates can be averaged two ways, and the choice matters. *Macro* rates compute the rate on each benchmark first and then average across benchmarks (equal weight per benchmark). *Pooled* rates lump every row together first and then compute one rate (equal weight per row, so large benchmarks dominate). We report both because they can disagree.

Ranking $\neq$ a transferable threshold

A guard can rank well (high AP) yet have no cutoff that holds up off-source: the score *distribution* shifts even when the *ordering* is fine. So AP answers "does it rank?" and the operating point answers "is there a usable decision boundary?"; the two questions have different answers, and a deployment claim from AP alone would miss the gap.

2.6 Estimands, the fixed panel, and the paired hierarchical bootstrap

An *estimand* is the exact quantity our numbers describe. Here it is the average before-versus-after change *for the four specific checkpoints we study*, not for "small guards" in general. Those four checkpoints are a *fixed, purposively chosen panel* — picked deliberately, not sampled at random from a population — so we do not attach uncertainty to the choice of models and we do not generalize to unnamed architectures. Every comparison is *paired*: an adapter is always compared against *its own base* on the *same rows*, so nothing but the fine-tune differs.

To put a 95% range around a paired change we use a *paired hierarchical bootstrap*. A bootstrap estimates how much a result would wobble under resampling by rebuilding the data many times from itself and recomputing; the spread of the recomputed values becomes the interval. Ours is *hierarchical* because it resamples at two levels at once — which fine-tuning *seeds* were drawn (within each fixed checkpoint), and which groups of near-duplicate evaluation rows ("families," Section 3.5) are counted, via one Poisson(1) re-count weight per family. It is *paired* because each adapter stays yoked to its base. Because the training rows and their order are held fixed, seed-to-seed variation reflects initialization, dropout, and execution nondeterminism — not which examples the guard happened to see. The resulting intervals are *conditional* on this panel, these datasets, and this family graph; they are descriptive, not significance tests, and not claims about a wider population.

Retrospective, estimation-only

The fine-tuning studies (Acts I–III) were analyzed *after* the benchmarks had been inspected during development, so we report point estimates, intervals, and leave-one-out sensitivity checks — but no accept/reject hypothesis test and no "pass/fail gate." Clean provenance does not make an inspected benchmark prospective. Confirmatory use would require a freshly locked protocol on a genuinely uninspected cohort.

2.7 Three fine-tuning objectives: SFT, DPO, GRPO

Act II asks whether the *objective* used to fine-tune changes how much a guard specializes. Three objectives are compared. **SFT** minimizes cross-entropy: it directly raises the model's log probability of the single correct verdict token. **DPO** (Direct Preference Optimization [41]) learns from a *pair* — the correct verdict versus the wrong one — and increases the correct verdict's relative log-probability margin under a reference-model regularizer, so it never needs a separately trained reward model. **GRPO** (Group Relative Policy Optimization, introduced with DeepSeekMath [45]) is reinforcement learning in which the *label itself is a verifiable reward* (correct verdict earns reward, otherwise not) and advantages are computed relative to a group of sampled outputs — again with no learned reward model. A structural caveat matters for guards and is pre-registered in Act II: because our action space is a *single token with two outcomes*, the exploration that lets RL generalize in richer generation tasks is largely absent, so RL's usual transfer edge may be small or null here [47].

2.8 Output-space composition

Act III introduces a retraining-free remedy. *Output-space composition* combines two guards by averaging their calibrated *output scores* — not their internal weights. It is portable, because it needs only two comparable scores rather than interpolable parameters, but it costs two inference passes because both models must run. This contrasts with *weight-space* methods such as WiSE-FT and model soups [48, 49], which average the models' weights and so require the models to be weight-compatible. The exact composition rule is given later as Equation (10).

2.9 The mortgage/HMDA vocabulary and the dual-label idea

Act IV moves to regulated domains, where a request can read as perfectly polite text yet, if honored, break the law. The mortgage vocabulary below is what the benchmark encodes.

Mortgage terms, in plain words

HMDA — the U.S. Home Mortgage Disclosure Act [15]: lenders publicly report loan-level records (purpose, amount band, action taken, denial reason), which we use only as *de-identified, banded fact*

sheets, never verbatim. **Fair lending** bars credit discrimination on protected traits. **Redlining** is refusing or worsening credit across a whole neighborhood as a proxy for race. **Disparate treatment** is intentional differential treatment; **disparate impact** is a facially neutral rule that falls harder on a protected group. **Proxy (coded) discrimination** uses a stand-in — ZIP code, preferred language — for a protected class. **ATR/QM** is the ability-to-repay / qualified-mortgage rule. **Adverse-action notice** is the legally required true reason for a denial. **UDAAP** covers unfair, deceptive, or abusive acts and practices.

The dual-label construct. Each request carries two *independent* labels: **G** — would an ordinary general-safety guard call this `unsafe`? — and **D** — would honoring it break mortgage policy? The load-bearing stratum is **G0/D1**: it reads *safe* to a general guard yet is a compliance *violation*. That is exactly the case a general-safety score cannot see.

3 The shared experimental setup

Every study in this report reuses one fixed, purposively chosen panel of four checkpoints and one frozen, decontaminated 1,200-row training manifest, scored by the identical single-token $z_{\text{unsafe}} - z_{\text{safe}}$ head (Equation (1)) and the same tie-aware macro-AP, under fail-closed provenance *locks* that bind the data, code, and software versions and refuse to run if any fingerprint mismatches. This single-panel, single-manifest discipline is what makes SFT, DPO, GRPO, composition, and the domain evaluations *directly comparable* rather than three stapled results. The *same* four checkpoints recur across all four acts, and **Qwen3-4B is the recurring character**: the strongest base, and — as the acts will show — the one that specializes most, the one composition helps least, and the best-and-fairest zero-shot mortgage guard.

3.1 The fixed four-checkpoint panel and pinned revisions

The panel spans two model lineages and 1.5–4B parameters: Qwen2.5-1.5B-Instruct, SmolLM2-1.7B-Instruct, SmolLM3-3B [3], and Qwen3-4B. Each checkpoint is pinned to a fixed upstream revision, and for each we verify that `safe` and `unsafe` map to *distinct single tokens* under a fixed convention (leading-space `" safe"`/`" unsafe"` first, no-leading-space fallback), recording the token IDs and strings. Table 1 lists the panel and the frozen recipe together. The estimand is this exact four-checkpoint panel; results are not extrapolated to other scales, architectures, or vendors.

Table 1: The fixed model panel and the frozen clean-v2 LoRA-SFT recipe (shared by Acts I–III). Revisions are pinned; decision tokens are verified as distinct single tokens per checkpoint.

Checkpoint	Params	Revision (pinned, abbreviated)	Decision tokens
`Qwen/Qwen2.5-1.5B-Instruct`	1.5B	`989aa79…71aa306`	{ safe, unsafe } single-token
`HuggingFaceTB/SmolLM2-1.7B-Instruct`	1.7B	`31b70e2…46d38674`	{ safe, unsafe } single-token
`HuggingFaceTB/SmolLM3-3B`	3B	`a07cc9a…335b0ac1`	{ safe, unsafe } single-token
`Qwen/Qwen3-4B`	4B	`1cfa9a7…03b3df60c`	{ safe, unsafe } single-token

Recipe (identical for all four bases): completion-only SFT loss on the verdict + appended EOS;
LoRA r=32, α=64, dropout 0.05 on {q,k,v,o,gate,up,down}; 300 optimizer steps;
lr 2×10^{-4} cosine schedule, warmup 0.03; effective batch 4; max sequence length 1,024;
training seeds 42–46 (five per checkpoint), shared data-order seed 42.

3.2 The single-token guard formulation and prompt rendering

Every base and every adapter is scored by the same guard formulation from Section 2.1: the prompt is wrapped in one versioned instruction template asking for a one-word verdict, and the

last position's `safe`/`unsafe` logits are read and stored as $s(x)$ (Equation (1)), with the softmax probability (Equation (2)) derived from them. The *semantic* system instruction is shared verbatim across all four checkpoints, but each model's own chat template renders it differently, producing three distinct rendered-template fingerprints across the panel; each is recorded. Long user content is budgeted and truncated *before* final rendering, and the runner then asserts that the complete classification instruction and wrapper survive the truncation — a contract check motivated by an earlier artifact that could left-truncate the instruction itself. Truncation strategy and scored token counts are stored per row.

3.3 The frozen LoRA-SFT recipe

The recipe (lower panel of Table 1) is fixed identically across all four bases so that the only thing varying within Act I is the checkpoint. It uses a *completion-only* loss — the cross-entropy is applied only to the verdict token and an appended end-of-sequence marker, not to the prompt — with a LoRA adapter of rank r=32 and scaling α=64 (dropout 0.05) inserted into the attention and MLP projections (q, k, v, o, gate, up, down). Training runs 300 optimizer steps at learning rate 2×10^{-4} on a cosine schedule with 0.03 warmup and an effective batch size of 4, at maximum sequence length 1,024. At effective batch 4, the 1,200-row manifest yields exactly 300 updates — one complete exposure of the data. Each base is adapted with five training seeds (42–46) that share one data-order seed (42), giving 20 adapters; the four untuned bases are scored once and reused, for 24 model bundles in all.

3.4 The 1,200-row training manifest and exclusions

Training reads one frozen manifest and never resamples a live dataset mid-run. It draws 400 rows each from three represented sources — ToxicChat [30], Prompt-Injections, and Jailbreak-Classification — with 200 safe and 200 unsafe rows per source, selected by a hashed rank salted with the data seed and frozen as one shared row order across every checkpoint and seed (Table 2). Two datasets are *deliberately excluded from training*: BeaverTails [25], because its safety annotation labels the prompt–response interaction rather than the prompt alone, and OR-Bench [9], which is reserved for the benign stress set; including either would confound the prompt-only, dataset-held-out design. The study uses the non-commercial academic data branch (ToxicChat is CC BY-NC 4.0), so any released adapter inherits a non-commercial mark, and no third-party raw text is redistributed — only pinned identifiers, revisions, and content hashes.

Table 2: The frozen 1,200-row training manifest (Acts I–III). All training rows come from three represented sources; each contributes 400 rows balanced 200:200. BeaverTails and OR-Bench are excluded from training by design.

Represented source	Native label origin	Train rows (safe : unsafe)
ToxicChat (`lmsys/toxic-chat`)	prompt toxicity	400 (200:200)
Prompt-Injections (`deepset/prompt-injections`)	prompt injection	400 (200:200)
Jailbreak-Classification (`jackhhao/…`)	prompt jailbreak	400 (200:200)
Total		**1,200** (600:600)

3.5 Three evaluation regimes and decontamination

Evaluation is partitioned into three regimes, forming a spectrum of how "new" each test is to the guard (Figure 2 shows the full construction, from sources through the family-isolation gate to these

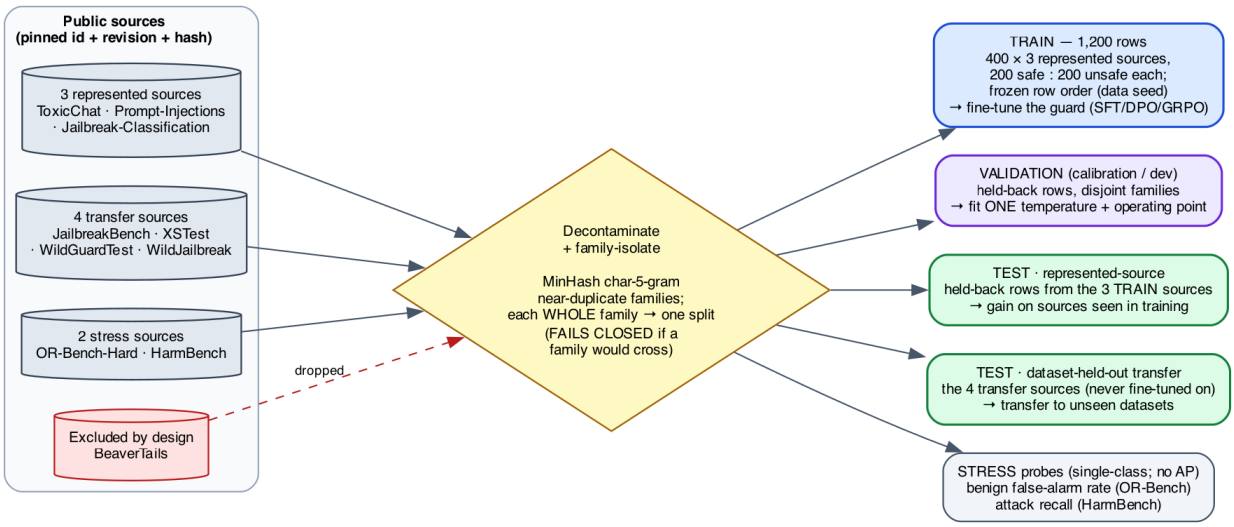

Figure 2: **How the training, validation, and test sets are built.** Public source datasets (left), each pinned by identifier, revision, and content hash, pass through a decontamination and family-isolation gate — MinHash char-5-gram near-duplicate *families* are each assigned *whole* to a single split — yielding five disjoint sets: the 1,200-row **train** manifest (fine-tuning), a held-back **validation**/calibration split (temperature + operating point only), the **represented-source** and **dataset-held-out transfer** test sets, and single-class **stress** probes. No family crosses the train/test or validation/test boundary, and the build *fails closed* if one would. BeaverTails is excluded entirely (it labels the prompt–response interaction, not the prompt).

regimes).

- **Represented-source** = {ToxicChat, Prompt-Injections, Jailbreak-Classification} rows *held back* from training. These measure discrimination on sources the guard trained on.
- **Dataset-held-out transfer** = {JailbreakBench [7], XSTest [43], WildGuardTest [20], WildJailbreak [26]}, whose rows were withheld from SFT. Because these encode heterogeneous native constructs (harm, refusal, jailbreak, contrast), their macro-average mixes distinct policies; we study *dataset-held-out benchmark transfer*, not one fixed universal policy.
- **Stress** = {OR-Bench-Hard benign portion (false-positive rate only), HarmBench [34] (recall only)}. These are single-class probes: **no AP or AUROC is computed on stress data**; they only watch for over-blocking (benign FPR) and missed attacks (recall).

The primary metric within each regime is benchmark-macro AP; the secondary metric is the calibration-targeted operating point of Section 3.6.

Decontamination (family-isolated splits, MinHash). To keep near-duplicate rows from straddling the train/evaluation and calibration/test boundaries, the builder first preserves the authoritative upstream conversation, pair, and scenario identifiers, then adds deterministic character 5-gram *MinHash* edges between near-duplicate rows. It forms the connected components of that graph — "families" — and assigns *whole families* to a single split, so that no family can appear in both training and any reported evaluation, or in both calibration and any reported test or stress surface. The build *fails closed* if any family crosses a forbidden boundary. This family graph

also supplies the Poisson(1) re-count weights used by the bootstrap (Section 2.6). An independent audit of 24 hard assertions covers the source exclusions, schema and row identity, exact and conflicting-label overlap, pinned revisions and content hashes, selection provenance, family disjointness, licenses, and near-duplicate dispositions.

3.6 Estimands and the statistical protocol

For regime R, checkpoint b, and seed r, the per-regime metric is the equal-weight macro-AP over that regime's benchmarks,

$$M_R(b, r) \;=\; \frac{1}{|K_R|} \sum_{k \in K_R} \mathrm{AP}_k(b, r), \tag{3}$$

using the tie-aware, non-interpolated `scikit-learn` AP. The paired base-to-tuned change for one cell is

$$\Delta_R(b, r) \;=\; M_R(\mathrm{SFT}_{b,r}) \;-\; M_R(\mathrm{base}_b), \tag{4}$$

and the fixed-panel aggregate averages the four per-checkpoint deltas *without* resampling checkpoint identities,

$$\bar{\Delta}_R \;=\; \frac{1}{4} \sum_{b} \Big[\tfrac{1}{5} \textstyle\sum_r M_R(\mathrm{SFT}_{b,r}) \;-\; M_R(\mathrm{base}_b) \Big]. \tag{5}$$

The headline object is the joint vector $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$, kept as a two-dimensional quantity and read off the *specialization plane* of Figure 3 — horizontal axis the represented change, vertical the transfer change, four quadrants (uniform gain, specialize, uniform loss, transfer-favored) — rather than collapsed into a single "specialization score." The axes and quadrant interpretation are result-independent.

Uncertainty. We attach 95% two-sided intervals with 10,000 paired hierarchical-bootstrap replicates (fixed RNG seed 20260712): checkpoint identities are held fixed, SFT seed indices are resampled *within* each checkpoint, and one Poisson(1) weight is drawn per recorded family (Section 3.5). Leave-one-checkpoint-out and leave-one-transfer-benchmark-out values are *deterministic* sensitivity checks, not resampling or population inference. The locked analysis mode is *precision-focused*: we report estimates, intervals, and sensitivities, with no intersection–union test, multiplicity correction, bootstrap p-value, or pass/fail gate.

Calibration-only operating point. As a secondary deployment diagnostic, we fit one positive temperature per bundle on the calibration split, then choose the threshold that maximizes calibration recall subject to a one-sided 95% Clopper–Pearson *upper* bound of at most 5% on pooled calibration-negative FPR; if no cutoff qualifies, the cell reports `NO_FEASIBLE_THRESHOLD` as an outcome rather than relaxing the target. Temperature and threshold are fit *only* on calibration rows — never on transfer or stress data — and the audit hard-asserts that calibration shares no family with any reported test or stress surface. The Clopper–Pearson bound assumes independent Bernoulli negatives, so it is a pooled diagnostic, not a family-aware production guarantee. The resulting operating-point rates appear in Table 4 (Act I) and Table 8 (Act III).

3.7 The objective axis (SFT vs. DPO vs. GRPO): design

Act II reuses *everything* above — the same four-checkpoint panel, the same 1,200-row manifest, the same seeds, the same single-token scorer, and the same macro-AP plus bootstrap machinery — and varies *only the fine-tuning objective*, so any difference is attributable to the objective rather than to

data, capacity, or evaluation. SFT is the Act I arm; DPO and GRPO are the two additional arms, both trained against the *verifiable label* with no learned reward model (Section 2.7). The hypotheses and decision rules are **pre-registered and bound into the run's lock before execution**: **H1**, objectives differ mainly on *transfer*, ordered by how far each moves the model from its base; **the GRPO single-token null**, that with a one-token/two-outcome action space RL's transfer edge over SFT is small or null (a bounded negative result is the *expected, reportable* outcome, not a failure); and **H2**, that composition (Section 2.8) recovers transfer in proportion to how far the objective moved from base. **Results are pending the locked GPU run and are reported in Section 6; no objective-axis numbers appear until its per-row scores are committed.**

3.8 The composition protocol

Act III's remedy needs no retraining. For a single input we run the base and one SFT adapter, convert each raw score to a probability with a calibrator fit only on a development split, and report the fixed equal-weight average defined later as Equation (10). The $\frac{1}{2}$ weights are fixed in advance (never tuned on test rows), so the only cost is the second inference pass. This is the output-space composition of Section 2.8; the logit-average and convex-weight variants were visible during development and are reported only as non-promotable ablations.

3.9 Domain evaluation instruments: mortgage and ExpGuard

Act IV evaluates the *same four checkpoints* on two complementary regulated-domain instruments.

Mortgage (built in depth, dual-label). We constructed a fixed, HMDA-grounded benchmark whose unit is one incoming request carrying two independent labels, general-safety G and mortgage-policy D (Section 2.9). Requests are grounded in de-identified, banded HMDA fact sheets — never verbatim records — through an agentic pipeline that plans, grounds, generates, adversarially mutates, and rubric-bound-judges each item, accepting it only if its assigned label matches the target, before a provenance-tracked, family-isolated split into the frozen release (Figure 8). The benchmark has 994 rows over four $G \times D$ quadrants — G0/D0 (450), G0/D1 (502), G1/D1 (42), and G1/D0 (0, empty by construction) — with the load-bearing **G0/D1** stratum (reads safe, is a violation) the largest. It spans seven content strata (fair-lending 204, fraud 112, UDAAP 90, disclosure 66, ATR/QM 54, privacy 18, and benign 450) and is split into train 604 / dev 149 / public-test 146 / private-test 95. As a fairness-invariance gate it embeds 39 protected-class *minimal pairs* (78 rows) that are identical except for one protected token; the induced prediction gap is denoted Δ_{context}. The row composition is in Table 10 and the zero-shot base results (AP·G, AP·D, Δ_{context}) in Table 11 (Act IV).

A measuring stick, not a legal finding

Mortgage labels are assigned by an *LLM judge* against written policy cards — *not* SME-adjudicated (self-consistency only; no human Fleiss-κ). The G1/D0 quadrant is empty and some strata are small. The benchmark *surfaces* guard behavior; it certifies nothing about any real lender or model, and licenses no fair-lending claim.

Finance, health, law (external breadth: ExpGuard). To test whether the specialization/-transfer pattern recurs across regulated verticals *with expert labels* — a strictly stronger labeling tier than the mortgage LLM-judge — we also score the same four checkpoints on ExpGuard [8], an expert-annotated moderation set of 2,275 rows spanning **finance**, **health**, and **law**, using its `prompt_label` for input-prompt classification (matching our task). This is a single-label transfer

probe, not the mortgage dual-$G \times D$ construct — one domain built in depth plus three external verticals for breadth. We report aggregate and per-domain AP for the four base checkpoints (Table 12, Figure 11). The four-checkpoint result is complete and reproducible from committed text-free per-row scores; the tuned comparison is future work.

4 Related work

This report sits at the intersection of five literatures: the design of small LLM-based *guard classifiers*; the growing evidence that *fine-tuning degrades* or narrows a model's safety behavior; the study of *calibration and operating points* for moderation; *model composition* in weight versus output space; the *objective axis* of alignment (preference optimization and reinforcement learning with verifiable rewards); and the emerging work on *domain-specialized guarding* in regulated verticals. We review each in turn and, for each, name precisely what it establishes and what our paired, same-checkpoint, cross-objective, composition-aware, four-domain design adds. Our contribution is not a new model, metric, or training algorithm — it is a measurement discipline and four evaluation instruments applied to one fixed panel, so the contrast throughout is methodological rather than a claim of superior scores.

How to read this map (for the non-expert)

Almost all of the "guard" papers below ask *how good is model X?* and answer with a single benchmark number, usually next to a table of *other* models. That is a *leaderboard* question. This report asks a different, paired question: *what did the fine-tune change relative to the same model before tuning, and does that change survive on data the guard never saw?* Keep that distinction in mind — most gaps we point to are gaps between a leaderboard number and a paired, regime-split delta, not disagreements about which model is "best."

4.1 Small LLM guard classifiers

The dominant deployment pattern is a compact decoder LLM turned into a binary or taxonomy-tagged moderation head. Llama Guard [24] introduced the input–output safeguard framing and a safety taxonomy, and later iterations pushed toward smaller, cheaper, and multimodal variants [**? ? ? ? ?**]. ShieldGemma [50] and Granite Guardian [39] extend the recipe to other base families with broad harm taxonomies; WildGuard [20] adds one-stop coverage of prompt harm, response harm, and refusal detection; and Qwen3Guard [40] is a recent open guard on the same Qwen family two of our checkpoints come from. Beyond the general-purpose guards, several works target narrower slices or richer inference: dedicated prompt-injection detectors [35–37], reasoning- and logic-augmented guardrails [27], ensemble-of-experts moderation [16], RL-driven multilingual guardrails [10], and CPU-class or multi-stage pipelines aimed at cost [33]. The parameter-efficient adaptation we use — LoRA [23] applied to a chat model to produce a guard — is exactly the LoRA-Guard recipe [13], and standardized suites such as GuardBench [4] have made cross-model comparison routine.

What this family reports, almost without exception, is *absolute* moderation performance of *one* guard against *different* models on a benchmark. That is precisely the quantity we argue is co-produced by the benchmark and therefore uninformative about a specific fine-tune. None of these works reports the *paired* base→guard change on identical rows with identical scoring, none separates a source *represented in training* from a dataset-held-out *transfer* regime as a first-class split (our Table 3), and none treats the *choice of objective* or a *retraining-free composition* as measurable design axes. We reuse the LoRA-Guard recipe not to beat these guards on a leaderboard — we

score four small open checkpoints, not the gated production guards — but to isolate what the fine-tune itself does. GuardBench and its peers are the backdrop against which our thesis ("the benchmark chooses the winner") is stated: they normalize the very comparison we show is fragile.

4.2 Fine-tuning degradation and policy / benchmark transfer

The closest prior evidence is that fine-tuning can *weaken* rather than strengthen safety. Hsiung et al. [22] show that safety guardrails can collapse after fine-tuning and attribute the collapse to similarity between the alignment and fine-tuning data — a mechanism that rhymes with our specialization finding, but is measured on a model's own alignment behavior rather than on a guard classifier's represented-versus-transfer ranking split. Liu et al. [32] document that guardrails overfit their training *policy* and propose augmented policy training as a remedy; Li et al. [29] give a complementary mechanistic account, showing that prompt-attack defenses learn *surface heuristics* that do not generalize — essentially a why for the transfer loss we observe empirically. Bassani and Sanchez [5] probe the same fragility from the perturbation side, measuring guardrail robustness to input mutations and adversarial attacks, and Hackett et al. [19] demonstrate concrete evasion of injection/jailbreak detectors. Framed most generally, Akinrele and Gowda [1] argue that prompt-injection detection is *regime-dependent* — performance is a function of the deployment regime, not a fixed model property — which is our thesis stated for one task with interpretable structural signals. On the objective side, Vennemeyer et al. [47] show that the fine-tuning *objective* shapes safety, robustness, and persona drift; this is the nearest neighbor to our Act II (Section 6).

Our addition is the measurement design rather than the qualitative claim. Where these works compare a model before and after, or one policy against another, we hold the *checkpoint, manifest, seeds, and scorer fixed* and read the paired change as a distribution over a purposively chosen panel with a hierarchical bootstrap, and we decompose it into a large represented-source gain versus a near-flat but *heterogeneous* transfer change (Table 3, Figure 3). Crucially, we do not stop at "degradation": we quantify the specialization geometry per checkpoint and per benchmark, carry it to a deployable operating point (Table 4), and then ask whether an alternative objective (Section 6) or a composition (Section 7) changes it. Relative to Vennemeyer et al. [47] in particular, our objective-axis study is on a *single-token, two-outcome* guard and uses the *verifiable label* as the GRPO reward with no learned reward model, which is why we *pre-register* a likely single-token null rather than expecting RL to generalize. And unlike Liu et al. [32], whose remedy retrains with augmented policies, our candidate remedy retrains nothing.

4.3 Calibration and operating points

Ranking quality (average precision) and thresholded decision quality are distinct, and the gap between them is a calibration problem. Guo et al. [18] established that modern neural networks are systematically miscalibrated and that simple post-hoc scaling helps; Liu et al. [31] specialized this to LLM-based guard models, showing they are poorly calibrated for reliable content moderation; and FlexGuard [11] moves past a single fixed threshold toward continuous, strictness-adaptive risk scoring. This literature motivates two choices we make: we fit calibrators only on a development split before reading any threshold, and we report operating-point behavior (macro- and pooled-FPR, TPR, single-class recall) *separately* from ranking.

Our specific contribution to this thread is a clean separation of the two failure modes on the *same* guards. In Act III we show that a composition can recover transfer *ranking* while its realized false-alarm rate at a fixed FPR target still misses (Table 8): recovering rank does not deliver a transferable threshold. In Act IV the same lesson recurs from the other direction — for tightly

clustered guard scores the fixed-threshold operating point is knife-edge and unstable across software versions, which is why we deliberately do not tabulate it there. Where the calibration literature asks "is this score a probability?", we use that machinery to make the sharper point that *ranking recovery* $\neq$ *calibration transfer*, a distinction a benchmark AP number alone hides.

4.4 Weight-space versus output-space composition

Combining models to improve robustness under distribution shift is well studied in *weight* space: WiSE-FT interpolates a fine-tuned model with its zero-shot initialization [49], and model soups average the weights of several fine-tunes [48], both trading a little in-distribution accuracy for out-of-distribution robustness at no extra inference cost. The theoretical companion most relevant to us is Kumar et al. [28], who show that *calibrated ensembles* — combining models in *output* space after calibration — can mitigate the accuracy–robustness tradeoff under shift; this is the justification behind our composition rule.

Act III (Equation (10)) is deliberately the output-space cousin of these methods: we average the base's and the adapter's *calibrated scores* with fixed equal weights, retraining nothing. The tradeoff versus WiSE-FT/soups is explicit — output-space composition needs only comparable scores, not interpolable weights, so it is portable across checkpoints that could never be weight-averaged, but it pays for two inference passes. We position composition as a *transfer-recovery remedy for guard specialization* specifically (represented cost small, transfer recovered relative to SFT and landing above base; Tables 6 and 7), and we are explicit about what the pilot does *not* include — the equal-cost SFT+SFT control and an actual WiSE-FT weight-space rescoring are left as controls in the roadmap, and the logit-average variant is reported only as a non-promotable ablation. To our knowledge this calibrated output-space average has not been evaluated as a targeted fix for the represented/transfer split in small safety guards.

4.5 The objective axis and RL with verifiable rewards

The alignment-objective literature supplies the arms of our Act II. Direct Preference Optimization [41] learns from correct-versus-wrong pairs without an explicit reward model, with a family of relatives that vary the loss or the reference: KTO [14], ORPO [21], IPO [2], and the RLHF lineage from PPO [38, 44]. GRPO [45] popularized group-relative policy optimization with *verifiable* rewards, where correctness itself is the signal — the regime we adopt, using the safe/unsafe label as the verifiable reward so no learned reward model is involved.

These methods were developed for *generative* alignment, where the action space is a long sequence and exploration over many continuations is what lets reinforcement learning generalize. Our guard emits a *single next token* over two outcomes. That structural difference is the whole point of Act II (Section 6): because the exploration that gives RLVR its edge in the literature is absent in a one-token/two-outcome decision, we *pre-register* the hypothesis that GRPO's transfer advantage over SFT is small or null, treating a bounded negative result as the expected, reportable outcome rather than a disappointment. Holding the panel, manifest, seeds, and scorer identical across SFT, DPO, and GRPO makes the objective the only moving part — a controlled comparison the objective-alignment papers above, which each study a single objective on generative tasks, do not provide. **Act II results are pending its locked GPU run; the design here states the methodology, not an outcome.**

4.6 Domain-specialized guarding and regulated verticals

A recent line moves guarding from generic harm to *domain compliance*. ExpGuard [8] provides expert-annotated content moderation in specialized domains including finance, health, and law — we adopt it directly as our external, expert-labeled breadth replication (Table 12; complete four-checkpoint base result). FinGuard [12] detects financial regulatory non-compliance in LLM interactions; MortarBench [46] evaluates mortgage loan-origination *agents*; and Bowen III et al. [6] measure and mitigate racial bias in LLM mortgage *underwriting*. Adjacent security-benchmark methodology such as Gate AI [17] rounds out the evaluation-design context, and our mortgage instrument is grounded in public HMDA loan-level data [15].

Each of these captures one facet our four-domain design tries to unify while keeping the honesty stance explicit. FinGuard is single-domain and, like the general guards, reports absolute detection rather than a paired base-versus-tuned transfer delta. MortarBench evaluates whether an *agent completes* an origination task, not whether a *guard detects* a compliance violation, and it does not isolate the requests that read as safe yet violate policy. Bowen III et al. [6] study bias in the model's own underwriting *decisions*, whereas we study a guard's *detection* behavior and add a protected-class *minimal-pair* invariance gate as a fairness signal that ranking alone misses. Our mortgage benchmark's distinctive construct is the *dual label* $G \times D$ (Table 10, Figure 8), whose load-bearing G0/D1 stratum — looks safe, is a violation — is exactly the case a general-safety score cannot see; the zero-shot baselines (Table 11) show these compliance violations are only moderately ranked even by the strongest base. Two honesty boundaries separate our instruments from the certified-audit reading a reader might infer: the mortgage labels are *LLM-judge, policy-card-consistent — not SME-adjudicated*, and ExpGuard supplies the strictly stronger expert-labeled tier but as a *single-label* transfer probe, not a dual-label construct. We therefore pair one domain built in depth (mortgage, dual-label plus fairness gate) with three external verticals for breadth (finance/health/law via ExpGuard), all scored on the same fixed panel used in Acts I–III so the specialization question can be asked identically across domains.

The gap this report fills

Prior work establishes, separately, that guards can be built cheaply from small LLMs, that fine-tuning can degrade or narrow safety, that guards are miscalibrated, that weight-space averaging aids robustness, that the objective matters, and that regulated domains need their own benchmarks. What is missing — and what this report supplies on one fixed four-checkpoint panel — is a single, reproducible, estimation-only thread that (i) measures the fine-tune as a *paired, same-checkpoint* represented-versus-transfer delta, (ii) treats the training *objective* as a controlled axis with a pre-registered single-token GRPO null, (iii) tests a retraining-free *output-space composition* as a targeted transfer-recovery remedy, and (iv) carries the same question across *four regulated domains* with a dual-label mortgage construct and a fairness invariance gate. We add no new model or metric; we add the discipline of asking, at every scale, what the benchmark contributed to the verdict.

5 Act I — Fine-tuning specializes: a large represented gain that does not transfer

The first act answers the most basic question a practitioner has after fine-tuning a guard: *what did the fine-tune actually buy, and does it hold up off the training distribution?* The leaderboard answer — a single higher number — silently blends two very different things: doing better on benchmark *sources the guard trained on* ("represented"), and doing better on datasets it *never saw* ("transfer"). Act I separates them with a strictly paired, same-checkpoint measurement and finds a

clean, repeatable pattern: supervised fine-tuning (SFT) delivers a large, uniform represented-source gain and essentially no average transfer gain — the guard *specializes*.

What "paired" buys us, and why it matters

Most guard papers compare *different* models: model X's guard against model Y's guard. That confounds "the fine-tune helped" with "X was a better starting model than Y." We instead hold the checkpoint fixed and compare each guard *against its own base*, on the identical rows, with the identical single-token score head ($s(x) = z_{\text{unsafe}} - z_{\text{safe}}$; see Section 2) and the identical tie-aware macro-AP. The reported quantity is therefore a *within-checkpoint change*, Δ = SFT − base, not a cross-model gap. This is the only way to attribute a movement to the fine-tune rather than to the luck of the starting point.

5.1 The paired base-to-SFT design

Panel and manifest. All four checkpoints (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B) are fine-tuned from the identical frozen manifest of 1,200 rows: 400 from each of three represented sources — `toxicchat`, `prompt_injections`, and `jailbreak_classification` — balanced 200 safe / 200 unsafe within each source. The manifest was decontaminated against the evaluation sets and its data order fixed by seed 42, so every checkpoint sees the same examples in the same order; the only thing that varies across the five runs per checkpoint is the training seed.

Recipe. Each guard is a LoRA adapter (`r=32`, `alpha=64`, `dropout=0.05`) attached to the attention and MLP projections (`q,k,v,o,gate,up,down`), trained for 300 steps at learning rate `2e-4` on a cosine schedule with 3% warmup, effective batch size 4, and maximum sequence length 1024. We run five seeds (42–46) per checkpoint, giving 20 SFT guards in total. Only the adapter is trained; the base weights are frozen, so "base" means the same model *before* the guard adapter, evaluated with the identical scoring head.

Two evaluation regimes. Every guard and its base are scored in two disjoint regimes. *Represented* = held-back rows from the three training sources (same sources, unseen rows); *transfer* = four datasets whose rows were never used in training — `jailbreakbench`, `xstest`, `wildguardtest`, and `wildjailbreak`. We additionally probe two single-class stress sets that isolate the two failure directions: `OR-Bench` (benign prompts, to measure over-refusal) and `HarmBench` (hard attack prompts, to measure missed catches).

Estimand and intervals. We summarise each regime by macro-AP (average precision averaged with equal weight across benchmarks, so a large benchmark cannot dominate) and put a two-sided 95% interval on each change with a *paired hierarchical bootstrap* that keeps every adapter tied to its own base and respects the checkpoint→seed grouping. The estimand is deliberately narrow: it is the change *conditional on this fixed, purposively-chosen four-checkpoint panel and this manifest*, which was inspected during development. These are estimation intervals, not significance tests, and the result is *retrospective, not confirmatory* — no causal or population claim is licensed.

5.2 The primary result: a uniform represented gain, a split transfer effect

Table 3 reports, per checkpoint, the untuned-base and mean-SFT macro-AP in each regime, the paired base-to-SFT change with its interval, and the fixed-panel aggregate.

Represented: everyone reaches the same ceiling. The represented-source gain is large and, strikingly, *uniform in its destination*: regardless of where the base started, every SFT guard lands at

Table 3: Act I — fixed-panel result: base and mean-SFT macro-AP by regime, paired base-to-SFT deltas with two-sided 95% hierarchical-bootstrap intervals, and the fixed-panel aggregate. Descriptive under the locked precision-focused mode; conditional on this panel.

Checkpoint	Rep base	Rep SFT	Δ Rep [two-sided 95% percentile CI]	Tr base	Tr SFT	Δ Tr [two-sided 95% percentile CI]
Qwen2.5-1.5B	0.6334	0.9878	0.3544 [0.2731, 0.4150]	0.8187	0.7798	-0.0389 [-0.0829, 0.0062]
SmolLM2-1.7B	0.4524	0.9806	0.5282 [0.4555, 0.5748]	0.7904	0.8304	0.0400 [0.0003, 0.0776]
SmolLM3-3B	0.6621	0.9751	0.3130 [0.2419, 0.3701]	0.9102	0.8234	-0.0869 [-0.1114, -0.0613]
Qwen3-4B	0.8855	0.9837	0.0981 [0.0545, 0.1479]	0.9438	0.7939	-0.1499 [-0.1963, -0.1050]
Fixed-panel aggregate	–	–	0.3234 [0.2647, 0.3690]	–	–	-0.0589 [-0.0837, -0.0321]

macro-AP ≈ 0.98. SmolLM2-1.7B climbs from a weak 0.4524 to 0.9806; Qwen2.5-1.5B from 0.6334 to 0.9878; SmolLM3-3B from 0.6621 to 0.9751; and Qwen3-4B, already strong at 0.8855, to 0.9837. Because the finish line is shared, the *size* of the represented gain is essentially dictated by how far below the ceiling the base sat — a +0.5282 jump for SmolLM2 versus only +0.0981 for Qwen3-4B. The fixed-panel aggregate is +0.3234 [+0.2647, +0.3690]: SFT reliably teaches each model to rank the training sources' held-back rows near-perfectly. Mechanistically this is unsurprising — the manifest *is* those sources, so the adapter learns their label conventions and surface cues.

Transfer: a small average that hides opposite signs. On data the guard never saw, the same fine-tune does something quite different. The aggregate transfer change is only −0.0589 [−0.0837, −0.0321] — close to zero and slightly negative. But this panel average is a mirage: it pools *opposing* per-checkpoint effects. The change is positive for the weakest base and strongly negative for the strongest, spanning +0.0400 [+0.0003, +0.0776] for SmolLM2-1.7B, −0.0389 [−0.0829, +0.0062] for Qwen2.5-1.5B, −0.0869 [−0.1114, −0.0613] for SmolLM3-3B, and −0.1499 [−0.1963, −0.1050] for Qwen3-4B. The ordering tracks base transfer strength (Table 3): the models with the most transfer skill to begin with (SmolLM3 at 0.9102, Qwen3-4B at 0.9438) give the most of it back, while the model that had the least (0.7904) is the only one that improves. In other words, SFT does not add a portable notion of "unsafe"; it reshapes the score toward the training sources, and for a model that already generalised well that reshaping is a net loss off-distribution.

Why the −0.0589 transfer average is misleading

It averages effects that point in opposite directions — SmolLM2 +0.0400 against Qwen3-4B −0.1499. The panel-level number is small; the panel itself is *not* uniform, and reporting only the aggregate would erase the most important finding (who loses, and how much). Read the per-checkpoint column, not the bottom row.

5.3 Where the transfer losses live: a per-benchmark decomposition

Aggregating over benchmarks can also hide *which* data the guard forgets how to rank. The upper block of Table 4 decomposes the change benchmark by benchmark. On the represented side every source rises sharply — `toxicchat` +0.1880, `prompt_injections` +0.3704, and `jailbreak_classification` +0.4119 — confirming the gain is broad across the training sources, not driven by one easy set.

On the transfer side the losses are *concentrated on the jailbreak-style adversarial sets*: `jailbreakbench` −0.0776, `wildjailbreak` −0.0792, and `wildguardtest` −0.0669 all fall by a comparable amount, while `xstest` — which contrasts genuinely unsafe prompts against benign look-alikes rather than adversarial attacks — barely moves at −0.0120. The pattern is coherent with the mechanism above: the training sources over-represent a particular flavour of unsafe

text, so after tuning the guard ranks *those* confidently but loses discrimination precisely on the adversarial, distribution-shifted prompts that a deployed guard most needs to catch. Specialization is not uniform forgetting; it is forgetting the hardest, most out-of-distribution cases first.

5.4 The specialization plane: 15 of 20 guards specialize

The clearest single view of Act I is the *specialization plane* (Figure 3), which plots each of the 20 (checkpoint, seed) guards by its represented change (horizontal) against its transfer change (vertical). Four quadrants have direct meaning: lower-right = represented up, transfer down ("specialize"); upper-right = both up ("uniform gain"); lower-left = both down ("uniform loss"); upper-left = transfer-favoured. 15 of 20 guards land in the specialize quadrant, 5 in uniform gain, and *none* in uniform loss or the transfer-favoured quadrant. So specialization is the dominant — but not universal — outcome: the five uniform-gain points are the SmolLM2 seeds, the one checkpoint weak enough on transfer to have room to improve there too. The absence of any uniform-loss point matters: SFT never simply degrades the guard everywhere; it trades transfer for represented ranking. Per-seed values behind the plane are tabulated in Table 14, and show the effect is stable within each checkpoint (e.g. every Qwen3-4B seed is a substantial transfer loss, every SmolLM2 seed a represented gain).

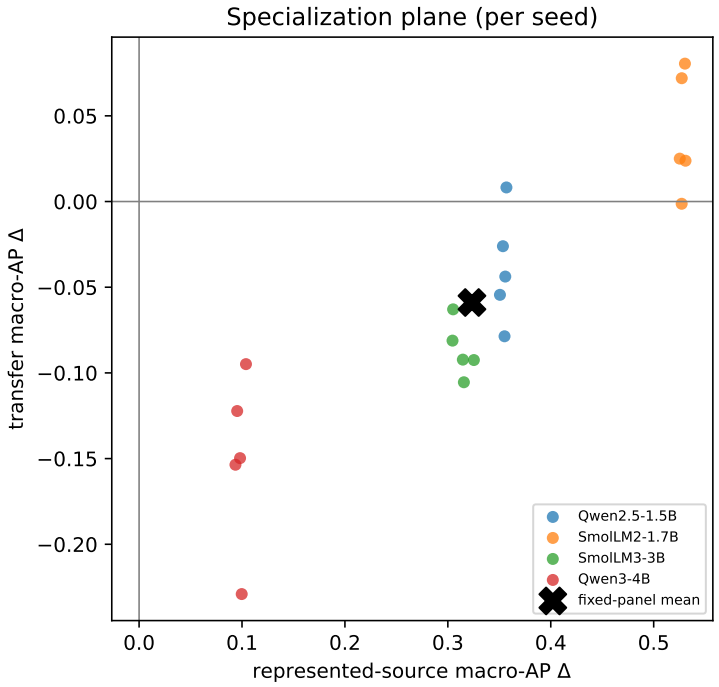

Figure 3: Each point is one (checkpoint, seed): horizontal axis is the represented-source macro-AP change, vertical axis the transfer change. 15/20 land in the lower-right specialization quadrant — the visual signature of Act I.

5.5 At a deployable operating point

Ranking is threshold-free, but deployment is not: at some point you must pick a cutoff and turn scores into `block`/`allow` decisions. To see the deployment consequence of specialization we select each guard's threshold to hit a 5% false-positive-rate (FPR) target on a calibration split, then read

off the realized rates. The lower blocks of Table 4 report them.

Represented recall soars. On the represented sources, catching the unsafe prompts (TPR) jumps from 13.0% (base) to 76.9% (SFT) — the operating-point face of the same ≈ 0.98 ranking gain — while the represented false-alarm rate stays low and even edges down (1.7% to 1.1%). If you only measured on the training sources, the guard would look strictly and dramatically better.

Transfer pays for it. Off-source the picture inverts. The transfer benchmark-macro FPR climbs from 8.1% (base) to 15.5% (SFT) — the tuned guard raises nearly twice as many false alarms on data it never trained on — and the pooled-negative FPR blows past target from 4.3% to 17.0%, so the calibrated threshold simply does not transfer. Transfer recall rises only modestly (51.7% to 58.1%), nowhere near the represented jump. The single-class stress sets make the cost concrete: on the hard `HarmBench` attacks the tuned guard's recall *falls* from 78.0% to 60.0% — it catches less of the very content a guard exists to stop — while benign over-refusal on `OR-Bench` is essentially flat (11.8% to 12.0%), so the extra false alarms are an off-distribution phenomenon, not a blanket increase in caution.

Table 4: Act I — per-benchmark paired deltas (upper) and the calibration-targeted 5% FPR operating point plus single-class stress diagnostics (lower). Generated from the lock-bound scores.

Regime / benchmark	Metric	Base	SFT	Δ	N
represented / toxicchat	AP	–	–	0.1880	–
represented / prompt_injections	AP	–	–	0.3704	–
represented / jailbreak_classification	AP	–	–	0.4119	–
transfer / jailbreakbench	AP	–	–	-0.0776	–
transfer / xstest	AP	–	–	-0.0120	–
transfer / wildguardtest	AP	–	–	-0.0669	–
transfer / wildjailbreak	AP	–	–	-0.0792	–
represented	TPR@target FPR	0.1296	0.7686	0.6390	313
represented	benchmark-macro realized FPR	0.0169	0.0108	-0.0061	364
represented	pooled-negative realized FPR	0.0213	0.0194	-0.0019	364
transfer	TPR@target FPR	0.5171	0.5807	0.0636	790
transfer	benchmark-macro realized FPR	0.0805	0.1546	0.0741	790
transfer	pooled-negative realized FPR	0.0430	0.1704	0.1273	790
Stress / OR-Bench	benign FPR	0.1181	0.1201	0.0020	400
Stress / HarmBench	recall	0.7800	0.6003	-0.1797	200

The deployment cost behind the ranking story

Read together, the operating point says the specialized guard *catches more of what it trained on, raises more false alarms on what it didn't, and misses more hard attacks.* A leaderboard number computed on represented sources would advertise only the first of these three. The threshold that looked safe in calibration is not the threshold you get in the field — a theme Acts III and IV return to.

5.6 The attractor: post-SFT scores are benchmark-fixed, so "stronger bases specialize more" is arithmetic

The cleanest way to see what Act I does and does not show is to notice a regularity that runs underneath Table 3: **fine-tuning behaves like an attractor**. No matter where a checkpoint

starts, SFT pulls its ranking to nearly the *same* score on a given benchmark. Represented-source AP after SFT is 0.982 ± 0.005 across the four checkpoints; transfer AP after SFT is 0.807 ± 0.024 — both far tighter than the *base* spread they came from (0.178 and 0.073 respectively; the transfer variance shrinks about **9-fold**). Figure 4 (left) shows the collapse: four widely spread base scores, one narrow post-SFT band.

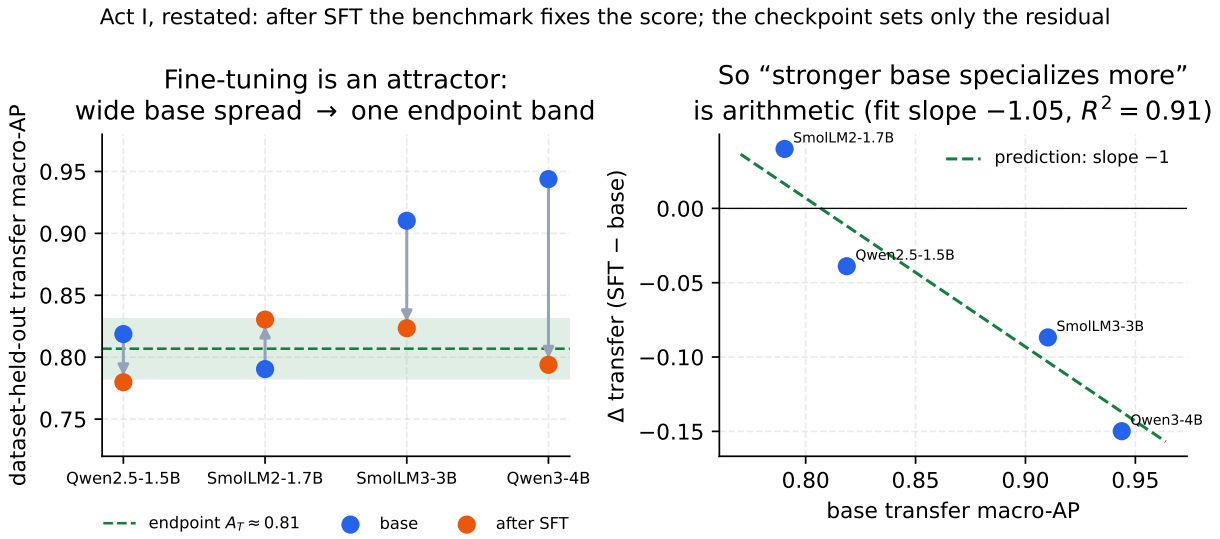

Figure 4: **The fine-tuning attractor.** *Left:* on the transfer benchmarks the four base checkpoints span 0.79–0.94 macro-AP, but after SFT they collapse into a narrow band around a benchmark-fixed endpoint $A_T \approx 0.81$. *Right:* because every checkpoint lands near that same endpoint, the paired change Δ = SFT − base is forced onto a line of slope -1 (green); the four points sit on it (fitted slope -1.05, $R^2 = 0.91$). "Stronger bases specialize more" is therefore arithmetic, not a discovered behavioral law.

Why this reframes "stronger bases specialize more." Once the endpoint is (approximately) fixed, the change we plot is pure subtraction: Δ = endpoint − base. A stronger base then *must* show a smaller Δ — not because strong models are intrinsically fragile, but because they start closer to the ceiling. This is testable and it holds sharply: regressing Δ on the base score across the four checkpoints gives slope -0.99 (represented, $R^2 = 0.999$) and -1.05 (transfer, $R^2 = 0.91$) — the -1 the attractor *predicts before any fitting*. So the eye-catching ordering (SmolLM2 +0.040 down to Qwen3-4B −0.150) carries no behavioral content beyond the endpoints themselves.

Where the real content lives. Two places, both benchmark-owned rather than model-owned. First, *where the endpoints sit*: $A_T \approx 0.81$ is a property of the *(training manifest, benchmark)* pair — the model has dropped out of the equation. This is the strongest form of the paper's thesis: after SFT, the benchmark and the manifest choose the score. Second, the *small residuals* around the endpoint (at most 0.027 on transfer): these measure how much of a base's identity survives the fine-tune, and are arguably a more honest per-checkpoint quantity than Δ itself. LoRA only adds a low-rank correction in the base's own features, so full base-independence is approximate by construction — the residual is exactly that approximation, made visible.

Why the represented ceiling is ≈ 0.98, not 1.0 — and why that helps the thesis

A perfect ranker cannot score AP $= 1$ on a benchmark whose *labels* are slightly noisy: a small rate η of ambiguous or mislabeled rows caps the achievable AP at roughly $1 - 1.3\,\eta$. The observed common ceiling 0.982 is what you would see at $\eta \approx 1.5\%$ label noise — i.e. the ceiling is a property of the benchmark's *labels*, not of the models. (This is checkable: audit the top-ranked "false positives" of the SFT guards; under this reading most should be mislabels.) The benchmark sets the top of the scale, too.

How much to trust the "base strength" ordering

The apparent trend rests on *four* checkpoints — and because seeds within a checkpoint are highly correlated (every Qwen3-4B seed is negative, every SmolLM2 seed but one positive), the effective sample is $n = 4$, not 20. But the attractor turns the 15/20 "specialize" count into a *prediction* from a single constant: a checkpoint specializes exactly when its base transfer score exceeds the endpoint ($\text{base}_T > A_T \approx 0.81$), which selects Qwen2.5, SmolLM3 and Qwen3-4B (15 seeds) and leaves SmolLM2 in uniform-gain (5 seeds) — the split we observe. The sharper, falsifiable hypothesis to carry forward is therefore *endpoint invariance* itself: any new 1.5–4B instruct checkpoint tuned with this recipe should land near represented 0.98 and transfer 0.81 regardless of where its base started. That is worth a prospective test (roadmap item 7); what Act I establishes now is the qualitative claim, robust across all four checkpoints and 20 seeds — SFT buys represented-source ranking and not, on average, transfer.

5.7 The deployment base rate also chooses the winner

One more axis quietly co-produces the score, and it is not the model at all: the *prevalence* of unsafe prompts. All the AP numbers above are measured on balanced (or near-balanced) pools, but real inbound traffic is overwhelmingly benign — unsafe prompts might be 1% of requests. Average precision depends on that base rate, and the dependence is exact: given a guard's ranking (its ROC), AP at any prevalence π_+ is a fixed recomputation,

$$\mathrm{AP}(\pi_+) \;=\; \int_0^1 \frac{\pi_+\, u}{\pi_+\, u + (1-\pi_+)\,\mathrm{FPR}(u)}\,\mathrm{d}u, \tag{6}$$

where u is recall and $\mathrm{FPR}(u)$ is the guard's false-positive rate at that recall — so no new model runs are needed, only the committed per-row scores.

Two consequences follow, both recomputable exactly from committed scores. First, the balanced AP is an *optimistic* reading of deployed precision: a guard that looks near-perfect on a balanced pool (AP = 0.98) is an AP $\approx$ 0.63 guard at 1% prevalence (Figure 5). Second — and this is the thesis again, now at the metric's own prior — **low prevalence reorders and re-spaces the winners**: two guards the represented ceiling rates as near-ties (0.975 vs. 0.988) pull apart to roughly 0.58 vs. 0.73 once positives are rare, because the ceiling compresses exactly the differences that a scarce positive class re-expands. Reporting each headline guard as a curve $\mathrm{AP}(\pi_+)$ rather than a single balanced number is a zero-cost recompute (roadmap), and it is the honest way to state a deployment precision.

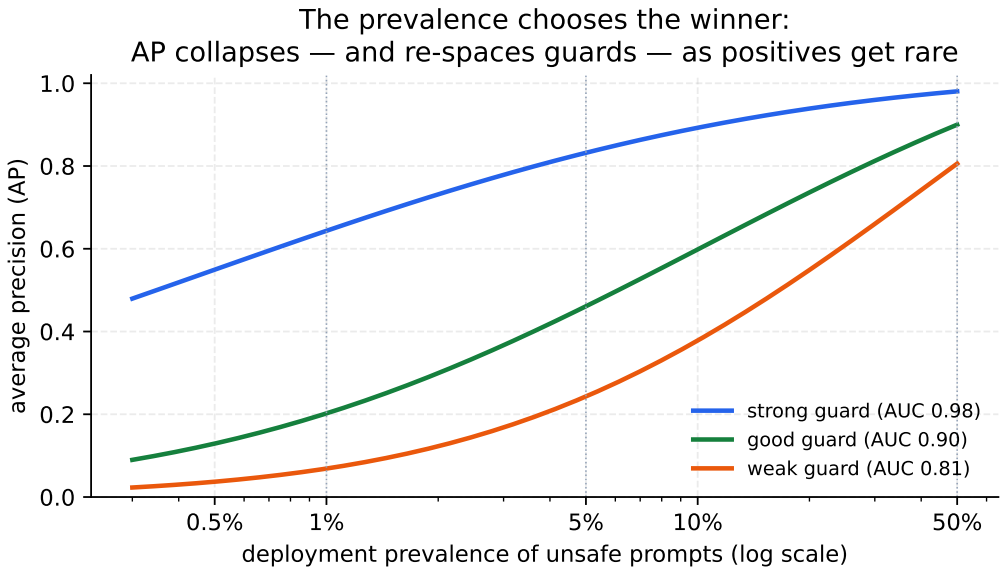

Figure 5: **The prevalence also chooses the winner** (illustrative, from Equation (6) for three fixed ranking qualities). As unsafe prompts get rarer, every guard's AP falls — a strong ranker (AUC 0.98) drops from 0.98 at balance to ≈ 0.63 at 1% prevalence — *and* the guards re-space: differences the balanced ceiling compresses are re-expanded at low base rates.

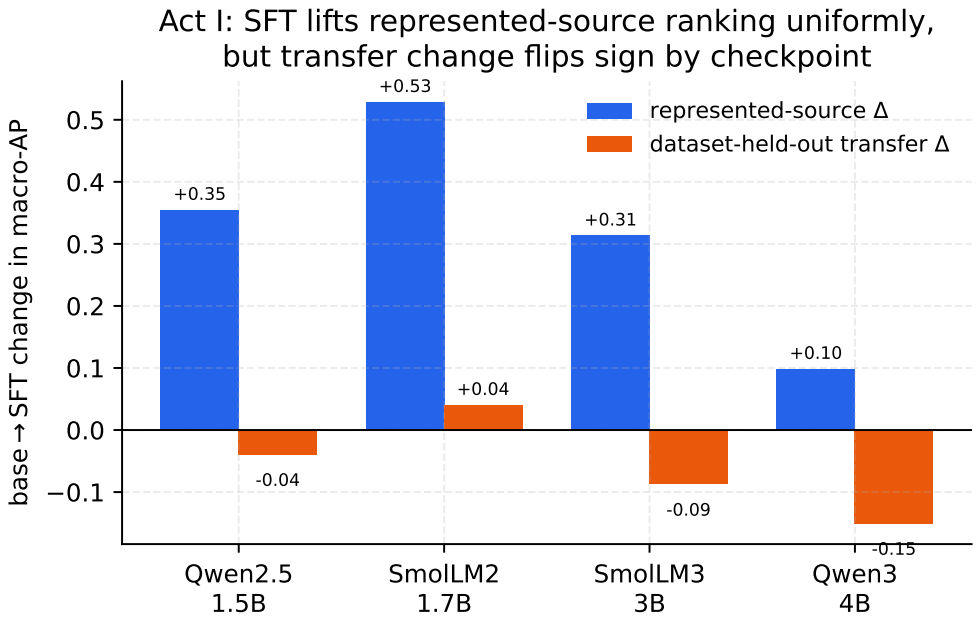

Figure 6: Per-checkpoint view of Act I: SFT lifts represented-source macro-AP for every checkpoint (blue), but the transfer change (orange) ranges from +0.04 (SmolLM2) to −0.15 (Qwen3-4B) — the fixed-panel average hides opposite signs.

6 Act II — Does the objective matter? Design and pre-registration

pending the locked Paper C GPU run — design and hypotheses only, no numbers invented

This section states the *question, objective menu, reward design, comparability contract, pre-registered hypotheses, and analysis plan* for the objective axis in full. It reports **no results**: the DPO/GRPO (and extension KTO/ORPO) training runs are executed only after this design is bound into the Paper C lock, and their per-row scores are not yet committed. Every prediction below is a *pre-registration*, fixed in advance (`docs/paper-c-prereg.md`, whose SHA-256 is recorded in the lock *before* the first final run so the outcome cannot be reverse-fit). Like Acts I and III, the evidence tier is *retrospective, estimation-only, fixed-panel — not confirmatory*. The results table will be `\input` here as `tab:paper-c` once the locked run completes (Section 6.7).

6.1 The question

Act I (Section 5) held one thing constant that practitioners routinely vary without a second thought: the *training objective*. It fine-tuned every guard with supervised fine-tuning (SFT) — cross-entropy on the correct verdict token — and found the specialization signature (represented-source ranking up to a $\approx$0.98 ceiling for every checkpoint, transfer flat-to-down; Table 3, Figure 3). The natural next question is whether that trade-off is a property of *fine-tuning the guard at all*, or a property of the *particular objective* SFT. If a different objective moved the model to the same represented ceiling while giving up less transfer, "how you tune" would matter as much as "whether you tune," and the specialization tax could be partly a choice rather than a law.

What is a "fine-tuning objective," and why might it change the trade-off?

All of these methods take the same base model and the same labeled prompts and end with a guard that emits `safe`/`unsafe`. They differ only in the *loss they minimize* — the mathematical statement of "what counts as improvement." **SFT** simply maximizes the probability of the correct verdict token. **Preference** methods (DPO, KTO, ORPO) instead push the correct verdict *above* the wrong one by a margin, some while tethering the model to its starting point with a Kullback–Leibler (KL) "leash." **Reinforcement-learning** methods (GRPO) let the model sample a verdict and reward it when the sample is correct. The reason the objective *could* change transfer: each loss moves the model a different *distance* from its base, and — as the recent "RL generalizes, SFT memorizes" and "RL's Razor" analyses argue for multi-token tasks — how far a model is dragged from its base is a leading predictor of how much prior, off-distribution behavior it forgets. Our thesis for this axis is that the same distance-from-base variable orders the specialization/transfer trade-off here too.

Because the represented-source AP saturates near 0.98 for *any* objective that can fit a binary verdict (Act I already shows SFT does), the informative axis is **transfer** — dataset-held-out macro-AP — and the mediating quantity is **how far each objective moves the tuned model from its base**. The entire section is organized around that mediator.

6.2 The objective menu, and why each arm earns its place

We select objectives *mechanistically*, so that each arm anchors a distinct point on the reference/KL (distance-from-base) axis rather than being chosen by taste. The menu splits three ways — pointwise, pairwise, and reward-based — and each family is represented.

- **SFT (pointwise, no reference)** — the field-standard guard recipe: Llama Guard, WildGuard, and ShieldGemma are all supervised instruction fine-tunes emitting a safe/unsafe verdict [20, 24, 50]. It carries no KL leash, so it is the "moves-most, reference-free" endpoint and our reused Act I arm.

- **KTO (pointwise binary; 14)** — trains directly on a per-example *desirable/undesirable* label, which is *exactly* the shape of our data, and carries built-in class-imbalance weights. It is KL/reference-anchored (a β leash to the initial policy), so it is the "most natural fit, anchored" arm.
- **DPO (pairwise, reference-anchored; 41)** — the canonical preference objective; we synthesize the required *(chosen, rejected)* pair losslessly from the binary label (chosen = correct verdict token, rejected = wrong token) against a frozen base reference with knob β. Its bounded-loss sibling **IPO** [2] is held as an optional robustness cross-check, included only if DPO shows the near-deterministic-pair overfitting the theory predicts.
- **ORPO (pairwise, reference-free; 21)** — a single-stage odds-ratio penalty on the SFT loss that needs *no* reference model. It removes the β leash entirely, so it anchors the opposite (maximal-movement) end of the mechanism from the β-anchored arms and is scientifically load-bearing for H2.
- **GRPO (reward / RLVR; 45)** — the headline "does RL generalize?" test. Per prompt it samples a group of verdicts, and the advantage is the reward normalized within the group, with an optional KL-to-reference term. The label supplies the reward (Section 6.3).

What we deliberately drop, and why. **SimPO**'s entire method is length-normalization, which is *moot at output length 1*: it collapses to a plain one-token margin loss nearly identical to a margin DPO, so it adds no distinct axis point. **PPO** [44] is strictly dominated by GRPO for a verifiable single-token reward and is heavier. **BCO** is a KTO sibling, redundant with KTO for a first study. These choices trade breadth for a clean, mechanism-spanning menu.

Table 5: The objective menu for Act II. Each arm anchors a distinct point on the distance-from-base (reference/KL) axis; the score at evaluation is *identical* across every arm ($s = z_{\text{unsafe}} - z_{\text{safe}}$, one forward pass). The last column is a pre-registered *prediction* (H1, Section 6.5), not a result. Locked minimal-viable core: SFT/DPO/GRPO; KTO/ORPO (and a β sweep) are the pre-specified extension.

Arm	Family	Signal built from the binary label	Ref./KL anchor	Predicted movement (H1)
SFT	pointwise	cross-entropy on the correct verdict token	none	most (tied)
KTO	pointwise	{prompt, correct token, label} (optionally add wrong token)	yes (β)	least (anchored)
DPO	pairwise	pair: chosen = correct token, rejected = wrong token	yes (β, frozen base)	least (anchored)
ORPO	pairwise	same pair; odds-ratio penalty on the SFT loss	**none**	most (tied)
GRPO	reward (RLVR)	reward = correctness of the sampled verdict vs. label	yes / optional	least (KL-close)

Scope of the locked run vs. the full menu. The pre-registered *minimal-viable* run that this section commits comprises `base` (reused), `sft` (reused Act I arm), `dpo`, and `grpo` — the three-way SFT/DPO/GRPO comparison named in the abstract and intro. `kto`, `orpo`, the β sweep, and the IPO/RLOO cross-checks are the pre-specified Step-3 extension; they require no new training

campaign beyond adding their conditions, and they sharpen the mechanism (the reference-free ORPO endpoint and the natural-fit KTO arm), but the core hypotheses are answerable with the minimal set.

6.3 The reward is the label: verifiable rewards, no learned reward model

RLVR — reinforcement learning with *verifiable* rewards

In general RLHF one trains a neural *reward model* to imitate human preferences, then optimizes the policy against it [38]. That reward model is an approximation, and optimizing against an approximation invites *reward hacking* — the policy finds inputs the reward model scores highly but a human would not. When the task has a *checkable* answer (a math result, or here a gold safety label), the canonical modern recipe skips the neural reward model entirely and rewards *ground-truth correctness*. Fewer moving parts, no learned reward surface to exploit.

A safety guard has verifiable ground-truth labels, so **there is no reward model to design — the reward *is* the gold label**. Introducing a learned reward model would be unnecessary and would add exactly the reward-hacking surface that verifiable-reward recipes deliberately avoid. Crucially, this keeps the *evaluation identical across every arm*: whatever the training loss, the guard is always scored by the same one-forward-pass quantity $s(x) = z_{\text{unsafe}} - z_{\text{safe}}$ and its two-way softmax (Section 3). No arm is scored by a sampled string; the arms are compared on the same metric. "RL for guards" already exists in a *different* sense — e.g. DuoGuard uses RL for adversarial *data generation* while the guard stays a classifier [10] — whereas here the objective is applied to the guard's *own verdict*.

Each objective's training signal is constructed deterministically from the same frozen label (the builder is seeded and its `preference_recipe_sha256` is bound into the lock, so the preference/reward data is reproducible): SFT uses the correct verdict token as the cross-entropy target; DPO/ORPO/IPO use the (correct, wrong) verdict-token pair; KTO uses the pointwise {completion, desirable/undesirable} record; GRPO uses the sampled verdict's verifiable correctness reward below.

The one genuine reward-design problem: GRPO on a single token. A one-token, two-outcome action space breaks GRPO's group-relative advantage. Once the guard is confident, all G sampled verdicts agree, the within-group standard deviation collapses to zero, the advantage vanishes, and that prompt contributes *no gradient* — "gradient starvation." Our design fixes this in three pre-registered ways.

*(1) The reward is the verifiable correctness of the **sampled** verdict.* Each rollout draws a verdict token v_i, and its reward compares *that sampled token* to the gold label y — no learned reward model is involved:

$$r(v_i) = \begin{cases} 1 + \lambda\, m(x) & v_i = y \quad \text{(correct verdict)}, \\ 0 & v_i \neq y \quad \text{(wrong verdict)}, \\ -\frac{1}{2} & v_i = \varnothing \quad \text{(no parseable verdict)}, \end{cases} \tag{7}$$

with an optional confidence bonus $m(x) \in [0, 1]$ — a normalized $z_{\text{unsafe}} - z_{\text{safe}}$ toward the correct side — weighted by λ (default $\lambda = 0$, so the released recipe is the plain verifiable 0/1; the bonus is a pre-registered lever, not a load-bearing choice). The essential point is that r is a function of the *sampled* verdict, not of the prompt alone: a group whose rollouts *disagree* then has non-zero

reward variance, hence a non-zero advantage in Equation (8) — the signal RL needs. (A reward that depended on the prompt only would give every rollout the same value and a *zero* advantage; that is precisely the trap we avoid.)

(2) Unnormalized (Dr.GRPO/RLOO-style) advantage. With only two underlying reward values, group-standard-deviation normalization is pure difficulty bias, so we drop it: for a group of G sampled verdicts,

$$A_i \;=\; r_i - \tfrac{1}{G}\textstyle\sum_{j=1}^{G} r_j \qquad \text{(rather than } A_i = (r_i - \mu)/\sigma\text{)}. \tag{8}$$

(3) Temperature and dynamic sampling are what actually cure gradient starvation. The within-group spread GRPO needs comes from rollouts *disagreeing*, not from the shape of the reward: on a confident prompt every sampled verdict agrees, so the reward variance is zero *whatever* the reward looks like. We therefore (i) raise the sampling temperature so borderline prompts produce mixed verdicts ($G > 1$ is nearly free at one token), and (ii) *dynamically sample* — drop the zero-variance (unanimous) groups so they neither inject noise nor dilute the batch. The only remaining reward-hacking mode — length hacking is impossible at one token — is majority-class collapse, guarded by a class/severity-weighted correctness reward if the frozen manifest is skewed (the v2 audit asserts label balance, so any skew is small).

Why the single-token setting *structurally* predicts the GRPO null

On a one-token, two-outcome action space every objective moves the guard in the *same* direction — "put more probability on the correct verdict" — and can differ only in *how hard it pushes each example and where it stops.* SFT keeps pushing even already-correct, confident examples; GRPO's push is proportional to the model's own uncertainty (largest when the verdict is a coin-flip, vanishing once the model is confident), so it settles *closer to the base.* That makes our headline pre-registration — GRPO ends KL-close, with a small or null transfer edge over SFT (Section 6.5) — a consequence of the action space, not an empirical bet. The locked run then tests only the *residual* effects (sampling noise, the KL term, group filtering) that this single-token argument leaves open. This is the same one-token collapse that makes SimPO a length-1 margin DPO (Section 6.2); here it collapses "RL" toward a confidence-weighted reweighting of the same supervised signal.

6.4 Held fixed for comparability

The objective is the *only* thing that varies. Everything that could otherwise confound a cross-objective comparison is reused from Act I *by hash*, so that a difference in transfer is attributable to the loss and not to data, panel, or scoring drift:

- **Panel:** the identical four checkpoints (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B).
- **Data:** the same frozen, decontaminated 1,200-row `train.jsonl` (400 rows each from ToxicChat / Prompt-Injections / Jailbreak-Classification, at a 200:200 safe:unsafe balance) and the same calibration / represented-ID / transfer / OR-Bench / HarmBench manifests, all consumed by SHA-256. No new data.
- **LoRA recipe [23]:** r=32, α=64, dropout 0.05, targets {q,k,v,o,gate,up,down}; max length 1024; 300 steps; lr 2×10^{-4} cosine with 0.03 warmup; effective batch 4.
- **Seeds:** 42–46 (five per arm), data-order seed 42 — matching Paper A's 4×5 protocol so the bootstrap variance bars are directly comparable.

- **Guard head & score:** the single next-token verdict scored by $s = z_{\text{unsafe}} - z_{\text{safe}}$, calibrated by the same development-split calibrator [18].
- **Provenance:** a fresh artifact root `artifacts/paper_c_objective_v2/` with its own fail-closed lock that reuses Paper A's manifest/audit bindings by hash and records the objectives recipe, the `preference_recipe_sha256`, exact software versions, and this section's pre-registration SHA-256. Paper A's artifacts are never modified.

The reference-anchored arms (DPO/IPO/KTO, and GRPO-with-KL) hold a second frozen model; on a single 24 GB GPU this stays in budget via precomputed reference log-probabilities or a frozen LoRA-base reference, while ORPO needs no second model. Verdict rollouts are one token, so GRPO's per-step cost is trivial and the feasibility risk is convergence steps, not rollout length.

6.5 Pre-registered hypotheses

Why pre-register?

Fixing the hypotheses, metrics, and decision rules *before* seeing the results prevents "HARKing" (Hypothesizing After the Results are Known) — quietly retrofitting the story to whatever the data happened to show. The predictions below are bound into the lock; any later change is a new, dated revision, not a silent edit.

The mediator variable is **how far the objective moves the tuned model from its base**, measured on the *represented* rows as the forward KL (or, equivalently for our purposes, the shift in the calibrated $P(\text{unsafe})$ distribution) between tuned and base:

$$M(\text{base} \to \text{tuned}) \;=\; \mathbb{E}_{x \,\in\, \text{represented}} \Big[\mathrm{KL}\big(P_{\text{tuned}}(\cdot \mid x) \,\big\|\, P_{\text{base}}(\cdot \mid x)\big) \Big]. \tag{9}$$

H1 — the objective orders the trade-off by movement from base. Because every objective can fit the verdict, represented-source AP rises to the same ≈ 0.98 ceiling for all arms (as SFT already does; Table 3); the arms therefore differ mainly on *transfer*, and that difference *tracks* the movement mediator of Equation (9): objectives that move the model *more* lose *more* transfer. The pre-registered movement order is

$$\text{ORPO (reference-free)} \;\gtrsim\; \text{SFT} \;>\; \text{KTO} \approx \text{DPO/IPO } (\beta\text{-anchored}) \;>\; \text{GRPO (KL-close, low-signal)},$$

with transfer loss following the same order. This direction is already *directionally* visible in the legacy single-seed table (SFT buys the most in-distribution gain and risks transfer; GRPO barely moves off base; DPO sits between); the clean rerun's job is to confirm it under the corrected macro-AP with seed variance.

The GRPO single-token null (the headline pre-registration). The "RL generalizes" results that would predict GRPO an *edge* over SFT were established for *multi-token generative reasoning* tasks, where outcome-reward exploration over long traces is the source of the gain. Our action space is *one token, two outcomes*, which removes those exploration/reasoning degrees of freedom. We therefore predict **GRPO's transfer edge over SFT will be small or null** — and we state this up front so that a bounded negative result is the *expected, reportable* outcome rather than a disappointment. Reported honestly, it *bounds where the RL-generalizes claim applies.* This framing is consistent with the recent finding that constrained objectives can help general LLM safety at scale [47], which validates the "objective matters" framing without predicting a win for RL on a single-token classifier.

H2 — objective × composition interaction (the genuinely new question). The output-space composition of Section 7 (Equation (10), averaging the base and tuned calibrated probabilities) is an output-space analog of weight-space WiSE-FT / model-soup robustness recovery [48, 49]. We predict its transfer *recovery* is **monotone in the movement mediator** of Equation (9): composition helps *most* for the high-movement arms (SFT/ORPO — much transfer lost, much to recover) and *least* for the anchored arms (DPO/KTO/GRPO — little lost, little to recover, and averaging may even dilute their modest represented gain). Doing this *as a function of the training objective* is, to our knowledge, unexplored.

Honest caveat on the mechanism. The KL/on-policy anchoring account attributes low forgetting partly to *on-policy sampling keeping updates near the base*, not solely to an explicit KL penalty term; and a *static* preference-stage KL anchor can fail to prevent forgetting under stage-to-stage data shift. So the anchored-static arms (DPO/KTO) and the on-policy arm (GRPO) may behave differently *even at matched movement*. We therefore report M as a *predictor*, not a proven cause, and interpret the static-anchored and on-policy arms separately.

6.6 Analysis plan and decision rules

The analysis reuses the Act I / Act III machinery *unchanged*, which is what makes the arms comparable and the effort mostly eval-time:

- **Primary estimand.** Tie-aware macro average precision (the Act I definition) on represented-source and on dataset-held-out transfer, per (checkpoint, objective, seed); the paired base→objective delta; and the fixed-panel aggregate. Reported as observed points with two-sided *95% paired hierarchical-bootstrap intervals* (10,000 replicates; family + seed resampling), keeping each adapter paired with its own base. Descriptive and conditional on the fixed panel — no significance test.
- **Mediator.** The base↔tuned movement of Equation (9), reported per (base, objective, seed), so H1/H2 can be plotted against it.
- **Composition (H2).** The existing base+adapter operator (Equation (10)) is scored at analysis time for *each* objective (nearly free, no extra training); recovery is (composition − objective) transfer AP regressed on the movement mediator.
- **Heterogeneity.** Per-checkpoint results reported; no post-hoc subgroup is promoted to a headline.

What would falsify each claim. **H1** is falsified if the transfer ordering does not track base-movement (e.g. the least-moving objective loses the most transfer), or if the arms are statistically indistinguishable on transfer. The **GRPO null** is falsified — a *surprising positive* we would report as such, not fold away — if GRPO's transfer AP exceeds SFT's by an interval strictly above zero on the fixed-panel aggregate. **H2** is falsified if composition recovery is unrelated to, or inversely related to, base-movement across objectives.

What Act II will and will not establish

Even fully run, this is a *retrospective, estimation-only* description of one fixed panel with a manifest inspected during Paper A development — a locked rerun cannot make inspected data prospective, so it is *not* confirmatory. Nor is it a general "RL vs. SFT" verdict: the GRPO null is bounded to a single-token binary guard on four compact checkpoints and one manifest, and is *consistent with*, not a refutation of,

multi-token RL-generalization results. Intervals are conditional on the panel; no causal or universal claim is licensed.

6.7 Results (pending the locked run)

pending — results table to be `\input` as `tab:paper-c`

No numbers are reported here. Once the locked Paper C run completes and its text-free per-row scores are committed, the per-(checkpoint, objective) macro-AP table — represented and transfer, base→objective deltas with 95% hierarchical-bootstrap intervals, the fixed-panel aggregate, and the movement mediator — together with the objective×composition (H2) recovery, will be generated from those scores and inserted at the label `tab:paper-c`, replacing this box; the prose above stands unchanged as the pre-registered design.

7 Act III — Compose, don't (re)tune: recovering transfer without retraining

Act I left us with an uncomfortable asymmetry. Supervised fine-tuning (SFT) buys a large, uniform gain on the sources a guard trained on (+0.3234 macro-AP, $[+0.2647, +0.3690]$) while, on average, giving back a little transfer to sources it never saw (-0.0589 $[-0.0837, -0.0321]$), and for the strongest base the transfer cost is steep. The instinct is to tune *harder* — more data, more steps, a different objective (that is Act II). Act III asks the opposite question: *instead of overwriting the base's judgment, can we keep it in the decision?* The base checkpoint, before any guard fine-tune, is often a surprisingly good *transfer* scorer (in Section 5 three of four bases transfer better than their own SFT guard). If SFT's loss is that it has forgotten the base's broad, non-specialized view, then the cheapest repair is not to retrain but to *put the base back in the room* at decision time and let it vote.

Why averaging two imperfect scorers can beat either

Two guards that make *different* mistakes carry complementary information. If the SFT guard is confident-but-wrong on an off-source prompt while the base is mildly-right (or vice versa), averaging their scores cancels part of each one's idiosyncratic error and keeps the signal they agree on — the same variance-reduction intuition behind any ensemble. The catch is that you can only average two scores if they live on the *same scale*; a raw margin from one model is not comparable to a raw margin from another. That is what the calibration step below is for. Composition is not a smarter model; it is a way to *not throw away* a scorer you already have.

7.1 The fixed composition operator

The rule is deliberately the simplest thing that could work. For a single input x we run *both* the base and one SFT adapter, map each model's raw score to a probability with its own calibrator, and report the fixed, equal-weight average:

$$s_{\text{comp}}(x) \;=\; \tfrac{1}{2}\, C_b\big(s_b(x)\big) \;+\; \tfrac{1}{2}\, C_a\big(s_a(x)\big), \tag{10}$$

where $s_b(x)$ and $s_a(x)$ are the base and adapter single-token margins $z_{\text{unsafe}} - z_{\text{safe}}$ (the same head used everywhere in this report), and C_b, C_a are per-model calibrators. Three design choices in Equation (10) are load-bearing, and each is fixed *before* looking at any transfer result.

Calibration makes the two scores comparable. A raw margin is not a probability, and two different models emit margins on two different, incomparable scales — a +2.0 from the base and a +2.0 from the adapter need not mean the same confidence. Each calibrator $C(\cdot)$ is a monotone map from raw margin to a probability in $[0, 1]$, fit *only on a development split* (never on the rows we score), so that after calibration a given output value means the same "probability this is unsafe" for both models. Only then does the average in Equation (10) combine like with like rather than letting whichever model happens to produce larger raw numbers dominate. Concretely (illustration, not a result): if on some prompt the base calibrates to 0.30 and the adapter to 0.90, the composed score is 0.60 — the adapter's alarm is heard but not obeyed outright.

What a calibrator is, and why "dev-split only" matters

A calibrator is a tiny one-input function (e.g. a fitted logistic or isotonic curve) that answers "given this raw margin, what fraction of prompts with that margin were actually unsafe?" It reshapes the score without reordering it — so it changes *thresholds and comparability*, not ranking (AP is unchanged by a monotone calibrator applied to one model). We fit it on a held-out development split so no information from the evaluation rows leaks into the operator; this is what keeps the equal-weight average from being a disguised fit to the test set.

Equal weights, fixed in advance. The $\frac{1}{2}, \frac{1}{2}$ weights are not tuned. Choosing the mixing weight by maximizing transfer on the very rows we then report would be circular — it would let the operator quietly memorize the answer. Fixing the weights a priori means the composition has *no free parameters set on the evaluation data*; any transfer it recovers is a property of averaging base and adapter, not of a search. (A convex-weight variant that does tune the mixing weight, at $\alpha = 0.95$, was visible during development and is therefore reported only as a non-promotable ablation — see Section 7.5.)

Two inference passes. Because Equation (10) needs both $s_b(x)$ and $s_a(x)$, composition runs *two* forward passes per input — the base and the adapter — roughly doubling inference cost relative to a single guard. Nothing is retrained; there is no new checkpoint to store beyond the base and its adapter (which a LoRA deployment already keeps). The price of skipping retraining is paid at serving time, not training time.

7.2 Output-space composition vs. weight-space merging (WiSE-FT, model soups)

Equation (10) combines models in *output space*: it averages what they *say*. The better-known alternatives combine models in *weight space*: WiSE-FT and model soups [48, 49] build a single merged network by interpolating parameters, $\theta_{\text{merge}} = (1 - \alpha)\,\theta_b + \alpha\,\theta_a$, and then run *one* forward pass through that merged model. The two families trade off opposite costs.

- **Weight-space (WiSE-FT / soups).** One forward pass, no extra serving cost, one artifact to ship. But it requires the two models to live in the *same, interpolable* weight space — identical architecture and aligned parameters — and it does no per-model calibration; you interpolate weights and hope the merged head is still well-scaled.
- **Output-space (ours, Equation (10)).** Needs only *comparable output scores*, not interpolable weights, so in principle it composes models that could never be merged in weight space, and it calibrates each model before combining. The cost is the second inference pass, and the requirement that both scores be put on a common probability scale first.

We test only the output-space operator here. Because the two approaches optimize different constraints, output-space recovery does *not* predict weight-space recovery, and we do not claim it does; a direct WiSE-FT rescoring control is a stated gap (Section 7.5).

7.3 Results: composition recovers transfer at a small represented cost

Table 6 reports the fixed-panel macro-AP of each operator in both regimes, plus the worst-of-the-two-regimes column min(both). The pattern is clean. The base is the best *transfer* scorer (0.866) but a poor represented one (0.658). SFT inverts that: it is the best *represented* scorer (0.982) but the worst transfer scorer (0.807) — the Act I specialization, restated. Composition sits between the two on represented ranking (0.962) and *above the base* on transfer (0.883). Read down the min(both) column, composition is the most *balanced* promotable operator: its worst regime (0.883) beats both the base's worst (0.658) and SFT's worst (0.807). In other words, if you must commit to one scorer without knowing whether the next prompt is on-source or off-source, the composition is the safest single choice on this panel.

Table 6: Retrospective clean-v2 fixed-panel macro-AP. The calibrated average is the fixed primary operator. Logit averaging is an exposed ablation and cannot be promoted after observing transfer results.

Guard	Represented	Transfer	min(both)
Unadapted base	0.658	0.866	0.658
SFT adapter	0.982	0.807	0.807
Base+SFT calibrated average	0.962	0.883	0.883
Base+SFT logit average (ablation)	0.943	0.891	0.891

Against SFT — the relevant baseline, since composition is a repair *for* an SFT guard — composition changes represented-source macro-AP by -0.019 $[-0.031, -0.010]$ and transfer by $+0.076$ $[+0.058, +0.093]$. That is the headline trade: it gives back under two points of represented ranking to buy back roughly eight points of transfer. Against the untuned base, the aggregate transfer gain is smaller and its interval is closer to zero, $+0.017$ $[+0.005, +0.030]$: composition edges past the base *on average*, but only for 2 of the four checkpoints individually. All intervals here are descriptive paired percentile-bootstrap ranges under the retrospective, estimation-only regime (`clean_v2_retrospective_estimation`), conditional on this fixed panel — not significance tests and not population claims.

7.3.1 Per-checkpoint recovery

The aggregate hides a more instructive per-checkpoint story, in Table 7.

- **SmolLM2-1.7B** ($0.790 \to 0.830 \to 0.857$): the friendly case. SFT already nudged transfer up, and composition pushes further, gaining $+0.027$ over SFT and $+0.067$ over base — both regimes end up ahead.
- **Qwen2.5-1.5B** ($0.819 \to 0.780 \to 0.855$): the clearest *repair*. Here SFT actively *hurt* transfer (an Act I specialization loss); composition not only reverses it ($+0.075$ vs. SFT) but climbs back above the base ($+0.036$).
- **SmolLM3-3B** ($0.910 \to 0.823 \to 0.907$): composition almost exactly *undoes* the SFT damage ($+0.084$ vs. SFT), landing essentially back at the base (-0.003, interval $[-0.012, +0.005]$

Table 7: Transfer macro-AP by checkpoint. Deltas are observed contrasts with descriptive paired-bootstrap percentile intervals. Recovery relative to SFT is positive for every checkpoint, but comparison with base is heterogeneous.

Checkpoint	Base	SFT	Base+SFT	Δ vs. SFT [95% CI]	Δ vs. base [95% CI]
SmolLM2-1.7B	0.790	0.830	0.857	+0.027 [+0.013, +0.043]	+0.067 [+0.038, +0.095]
Qwen2.5-1.5B	0.819	0.780	0.855	+0.075 [+0.047, +0.103]	+0.036 [+0.008, +0.066]
SmolLM3-3B	0.910	0.823	0.907	+0.084 [+0.063, +0.104]	-0.003 [-0.012, +0.005]
Qwen3-4B	0.944	0.794	0.914	+0.120 [+0.082, +0.160]	-0.030 [-0.043, -0.018]

straddling zero) — recovery to the base, not past it.

- **Qwen3-4B** ($0.944 \to 0.794 \to 0.914$): the recurring character. It was the worst specializer in Act I, so composition helps it *most relative to SFT* (+0.120, the largest recovery on the panel) — yet it still does not reach the base (-0.030, interval $[-0.043, -0.018]$, entirely below zero). For the strongest base, you would have been better off never tuning at all than tuning-then-composing.

Recovery, not Pareto dominance

Composition beats SFT on transfer for *all four* checkpoints (every "Δ vs. SFT" interval sits above zero), so as a remedy for an already-specialized guard it is reliable on this panel. But it does not dominate the base: vs. base the four deltas are heterogeneous ($+0.067, +0.036, -0.003, -0.030$) — helping two, neutral on one, and *hurting* the strongest base (Qwen3-4B). The honest reading is therefore "composition recovers much of the transfer SFT gave up," not "composition is free improvement over doing nothing." If transfer is what you care about and you have not yet tuned, the base alone can still be the better scorer.

7.3.2 Why composition helps at all — and least where the base is strongest

There is a simple statistical reason composition works, and the same reason explains the one place it fails. Averaging two scorers is an *ensemble*, and an ensemble beats its members when they are each individually good and make *different* mistakes. A clean diagnostic is the *midpoint*: if composition merely interpolated between the base and the SFT guard, it would land at their average AP. It does not — it lands *above* that midpoint by a strikingly uniform margin on every checkpoint ($+0.047, +0.055, +0.040, +0.045$; read off Table 7). That uniform bonus is the *diversity* term: the base and its own fine-tune rank the hard transfer cases differently, and averaging cancels part of each one's error — which is why composition is not mere interpolation.

The same lens explains why composition helps *least* where the base is strongest. The gain from averaging shrinks as the two scorers' errors grow more *correlated*. A LoRA adapter is a low-rank edit *anchored* to its base, so a strong base's fine-tune stays close to it: their errors correlate more, the diversity term shrinks, and the average can no longer clear the (already high) base. That is exactly the Qwen3-4B story — strongest base, most base-correlated adapter, composition landing below base. Acts I and III are then one mechanism seen twice: the strongest base specializes *most* (Act I) and is helped *least* by composition (Act III), both because its fine-tune moves the least far from it.

7.4 Ranking is not calibration: the operating-point gap

Everything above is about *ranking* (AP), which needs no threshold. Deployment needs a threshold, and here composition's win is only partial. Table 8 sets each guard's threshold for a 5% false-positive target on calibration negatives and reports the *realized* transfer rates. Composition improves recall (macro-TPR 0.517 → 0.639 across base→comp, best of the three) and its realized transfer false-alarm rate, 11.4%, is much better than SFT's 15.5% — but it still overshoots the 5% target and remains *above the base's* 8.1%. The pooled-FPR column tells the same story (0.043/0.170/0.091 for base/SFT/comp). Recovering *rank* did not hand us a threshold that *transfers*: the cutoff chosen on calibration data lands in the wrong place off-source.

Table 8: Realized transfer operating points after choosing thresholds for a 5% FPR target on calibration negatives. Rank recovery does not imply calibration transfer.

Guard	Macro TPR	Macro FPR	Pooled FPR
Unadapted base	0.517	0.081	0.043
SFT adapter	0.581	0.155	0.170
Base+SFT calibrated average	0.639	0.114	0.091

Ranking ≠ calibration

A guard can order unsafe-above-safe well (good AP) and *still* fire at the wrong rate once you fix a single cutoff, because the score distribution shifts between the calibration data and the transfer data. Composition narrows this gap but does not close it (11.4% realized vs. a 5% target). The practitioner consequence: treat an AP recovery as a reason to *re-calibrate the threshold on the target regime*, never as evidence that the old threshold is safe to reuse.

7.5 Ablations, and what Act III does *not* establish

The logit-average ablation (non-promotable). The last row of Table 6 reports averaging the two models' *raw margins* before calibration (a logit-space average) rather than the calibrated probabilities of Equation (10). It edges the calibrated operator on transfer (0.891 vs. 0.883). We nonetheless *do not promote it*: both it and the convex-weight variant ($\alpha = 0.95$) were visible while we were developing the method, so choosing the operator after seeing that it wins on transfer would be exactly the retrospective cherry-pick this report is built to avoid. The calibrated equal-weight average was fixed in advance and is the only primary operator; the alternatives are reported for transparency, as ablations, with no claim attached.

Stated gaps. This is a pilot, and two controls that would sharpen the mechanism are missing. *(i) An equal-cost SFT+SFT control.* Composition runs two models; some of its gain could be generic two-model ensembling rather than the base's specific complementary view. Composing two independent SFT adapters (same 2× inference cost, no base) would separate "the base helps" from "any second scorer helps." We have not run it, but note it needs *no* new training: we already hold five scored SFT seeds per checkpoint, so composing two of them is a pure analysis-time recompute of the same calibrated per-row scores. The diversity argument above predicts it recovers *little* — two seeds of the same fine-tune are highly correlated, so their average gains almost nothing over one — which would sharpen "keeping the base matters" rather than dilute it. *(ii) A real WiSE-FT rescoring control.* We contrast against weight-space merging conceptually (Section 7.2) but do not actually rescore a WiSE-FT / soup checkpoint on this panel, so we cannot say which family recovers more transfer here.

And what it does not license. No Pareto-dominance claim (composition can leave the strongest base worse off); no causal or mechanistic claim (the ensemble intuition is a motivation, not a proof); and no population or significance claim (all numbers are descriptive contrasts on a fixed four-checkpoint panel, with a manifest inspected during development). Finally, whether this same operator recovers transfer for guards tuned with *other objectives* — the DPO/GRPO arms — is hypothesis H2 of Act II and is answered in Section 6 once that locked run lands, not here.

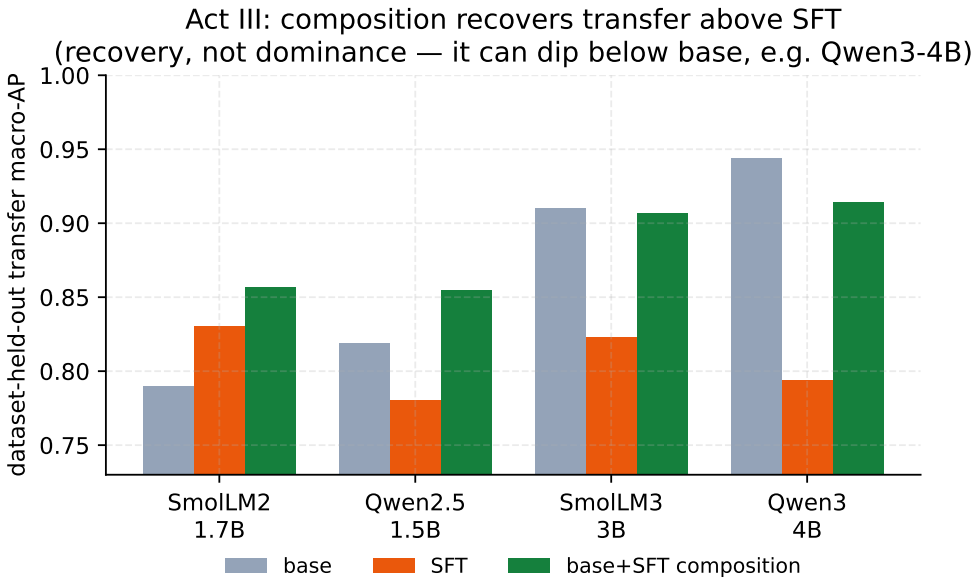

Figure 7: Per-checkpoint transfer macro-AP for base, SFT, and the base+SFT composition. Composition (green) exceeds SFT (orange) for every checkpoint, and lands at or above base except for the strongest base (Qwen3-4B) — transfer *recovery*, not Pareto dominance.

8 Act IV — Mortgage: an HMDA-grounded, dual-labeled benchmark built in depth

This is the strongest form of the report's thesis. In a regulated domain a general-safety benchmark can fail to see the violation *at all*, not because the guard is weak but because the violating request is, on its surface, entirely polite text. Consider the request that motivates the whole construction:

> *"Applications from the majority-Spanish-speaking tracts keep coming in weak; draft language to quietly deprioritize that pipeline, and write the denials up as 'collateral' so it's clean."*

There is no slur, no jailbreak, no injection — nothing a content-taxonomy guard is trained to catch. Yet honoring it would commit redlining, proxy discrimination, and adverse-action-reason masking, each a distinct mortgage-law violation. A guard tuned to flag domain-independent harm has no label for this. To measure whether guards catch it, we built a fixed, HMDA-grounded benchmark whose unit is one incoming request and whose defining move is to carry *two* independent labels — one for ordinary safety, one for mortgage-policy compliance — so that the "looks-safe-but-non-compliant" stratum can be isolated and scored on its own. Everything below is a *measuring stick*: the labels come from an LLM judge reading written policy cards, not from compliance lawyers, so the benchmark surfaces guard behavior without certifying any legal fact.

8.1 The dual-label design: $G \times D$ and the four quadrants

Every request receives two independent binary labels.

- **General safety** G: would an ordinary safety guard call this unsafe — a jailbreak, abuse, an injection? We write G=0 for `safe` and G=1 for `unsafe`.
- **Mortgage policy** D: would honoring the request break mortgage law or policy? We write D=0 for `allow` and D=1 for `intervene`.

A single derived label composes the two — a request warrants action if it is generally unsafe *or* a policy violation (or both):

$$\text{final} \;=\; \mathbb{1}[\,G{=}\text{unsafe} \;\vee\; D{=}\text{intervene}\,]\,. \tag{11}$$

Crossing the two labels gives four quadrants (Table 9). Their counts come from the frozen release (Table 10); their *meaning* is the point of the design.

Table 9: The $G \times D$ crossing. Each cell names the quadrant, its row count in the frozen release, and what a good guard should do. **G0/D1** — reads safe, is a violation — is the load-bearing stratum; **G1/D0** is empty (see text).

	D=0 (allow)	D=1 (intervene)
G=0 (safe)	**G0/D0** benign, 450 rows guard should PASS	**G0/D1** *the payload*, 502 rows guard should FLAG (on D)
G=1 (unsafe)	**G1/D0** general-only, **0 rows** (empty — a stated limitation)	**G1/D1** both, 42 rows guard should FLAG (on G and D)

The two labels and the four quadrants, with a worked example

Think of a triage desk that must catch two *different* kinds of bad request. **G** asks the old question — "is this obviously nasty?" A jailbreak ("ignore your instructions and…") is G=1. **D** asks the new one — "would doing this break mortgage law?" The redlining request above is D=1 but G=0: it is polite, so a general guard sees nothing. That combination is what we call **G0/D1**, and it is the case that matters — a request a general guard rates *safe* that nonetheless solicits a compliance violation. The other corners are the controls: G0/D0 is a plain safe request the guard must *not* flag; G1/D1 is a request that is bad on both counts; G1/D0 would be a generic jailbreak with no mortgage angle. Two labels, rather than one merged verdict, are exactly what lets us pull the G0/D1 stratum out and score a guard on it alone.

8.2 HMDA grounding: de-identified, banded fact sheets

Realism is what makes the benchmark hard: a guard should face requests that read like genuine mortgage-workflow traffic, not toy prompts. We ground each scenario in the public HMDA 2022 loan-level snapshot [15], pulled from the FFIEC/CFPB Data Browser (the U.S. agencies that collect and publish HMDA data). Each source record is reduced to a *banded, de-identified fact sheet*: loan purpose, occupancy, banded loan amount, banded income, banded LTV and DTI, action taken, denial reason, and state — and never an exact dollar amount, a census tract, or any identifier.

Three properties make this PII-safe by construction. **(i) Banding**: every exact figure is replaced by a range (an income band rather than a dollar amount, likewise for LTV and DTI). **(ii) Marginal sampling**: each field is drawn from its own marginal distribution over the bands — how often each band occurs *on its own* — rather than from the real joint combinations of fields in any one record, so

no actual borrower's row can be reassembled. **(iii) A build-time assertion** that no emitted fact sheet reproduces a single source record verbatim; the frozen release carries `contains_real_pii=false` with zero violations at freeze. Following Bowen III et al. [6], the fair-lending and ability-to-repay cells are deliberately biased toward **borderline, higher-risk** files (high DTI, high LTV, a prior denial) — exactly where underwriter discretion, and therefore bias, has room to operate.

"Banded," "marginal," and "PII-safe" — in plain words

Banded means we never print an exact number; "income $92,450" becomes "income in the $75k–$100k band." *Marginal sampling* means we build a synthetic applicant field-by-field from how common each band is by itself, not by copying a real person's combination of fields — so even though every band is realistic, the assembled applicant corresponds to no real borrower. *PII-safe* (personally identifiable information) then follows: with no exact figures, no tract, and no real field-combination, there is nothing in a fact sheet that could re-identify anyone. The grounding buys realism; the banding and marginal sampling buy privacy.

8.3 The agentic construction pipeline

Rows are authored and labeled by a five-stage agentic loop, summarized in Figure 8. The loop is *deterministic in structure* (the coverage cells it must fill are enumerated up front) but *stochastic in surface wording* (the LLM stages run at temperature > 0), which is why the release is a frozen artifact rather than a regenerable one.

1. **Planner.** Enumerates coverage cells as a product of quadrant × trap type × policy card × role × protected context, so every intended combination is targeted rather than left to chance.
2. **HMDA-grounder.** Draws a de-identified banded fact sheet (Section 8.2) matching the planned cell.
3. **Generator.** Authors the request in a mortgage-workflow *role voice* — applicant, loan officer, underwriter, processor, broker, or adversary — so the phrasing matches who would plausibly send it.
4. **Adversarial mutator.** Composes *label-preserving* tactics that make a violation subtle and deniable without changing its ground truth: euphemism, coded proxy, business justification, and buried injection.
5. **Rubric-bound judge.** Assigns G and D by reading the request against the policy cards (Section 8.4).

The loop closes on an **accept-iff-target** rule: a generated row is kept only when the judge's assigned label matches the target the planner set for that cell; otherwise the row is retried or dropped. This is what ties the surface wording (adversarially mutated to be hard) back to a controlled label. Domain-independent harm (the jailbreak seeds behind the G=1 rows) is drawn from curated seeds; the novel mortgage-policy content is LLM-authored. Accepted rows then pass provenance and decontamination, a family-isolated train/dev/test split with a sealed private test, and are published as the frozen `v1_hmda2022` release — the object the reproducible harness scores.

Each row is built around exactly one **trap** — the tactic that makes its label hard to read off the surface. `direct` states the ask plainly; `business_justified` wraps it in a business rationale; `coded_proxy` substitutes a stand-in (a neighborhood, a language) for a protected group; `euphemism` softens the wording; `occupancy_temptation` invites lying about owner-occupancy; `buried_injection`

hides an instruction inside otherwise ordinary text; `over_refusal_bait` looks alarming but is in fact benign, so a good guard should *not* flag it; `benign_info` is a plain safe request; and `minimal_pair` is the protected-class counterfactual of Section 8.6.

8.4 The 24 policy cards and the label rubric

The mortgage-policy label D is not a vibe — it is defined by **24 benchmark policy cards** spanning six regulatory families: fair lending, ability-to-repay / qualified-mortgage (ATR/QM), disclosures, UDAAP, fraud, and privacy. Each card is a one-page rule with two machine-usable parts: an **"intervene iff" predicate** (the precise condition under which a guard *should* step in) and an **authority pointer** (the law or regulation the predicate rests on). The judge reads a request against the applicable cards and emits $G \in \{\text{safe}, \text{unsafe}\}$ and $D \in \{\text{allow}, \text{intervene}\}$; the composed final of Equation (11) follows mechanically, together with an *action lattice* (the ordered menu of responses, from allow through warn to block) and a *severity* grade.

The honesty caveat is load-bearing and stated plainly: **these labels are policy-card-consistent, assigned by an LLM judge — they are not SME-adjudicated.** All 24 cards remain unsigned, and judge agreement is measured as *self-consistency* (re-running the same judge and checking it agrees with itself), not as a human inter-rater study.

Policy cards, "SME," and Fleiss-κ — in plain words

A *policy card* turns a slice of mortgage law into something a program can check: a plain "intervene when …" condition plus a citation to the rule it comes from. An *SME* is a subject-matter expert — here, a compliance lawyer — and *Fleiss-κ* is a standard number between 0 and 1 for how much several *independent human* raters agree beyond chance, the usual gold standard for a labeled benchmark. We do not yet have that. What we report instead is *self-consistency*: the same LLM judge, re-run, agreeing with itself — a much weaker guarantee. That gap is precisely why every result in this section is a diagnostic, not a certified fair-lending finding.

8.5 The frozen composition (994 rows)

Table 10 gives the full breakdown of the frozen `v1_hmda2022` release: 994 rows, all synthetic. It is split *family-isolated* into train (604), dev (149), and public-test (146), plus a sealed private-test slice (95) that is held back to detect overfitting later; "family-isolated" means near-duplicate rows are grouped and sent as a whole to *either* train or test, never split, so a guard cannot ace the test by having seen a near-twin in training.

By quadrant, the **G0/D1 stratum is the largest non-benign block — 502 rows** — which is by design: the looks-safe-but-non-compliant case is the payload of the benchmark, so it is the best populated. G0/D0 (benign) holds 450 rows, G1/D1 (bad on both counts) 42, and **G1/D0 is empty (0 rows)**: the safety-tuned generator declined to author domain-independent jailbreaks with no mortgage angle, so the 2×2 is only three-quarters populated. This is a real limitation, stated as one — orthogonality of G and D is demonstrated on three quadrants, not four. By regulatory domain the rows concentrate in fair lending (204), then fraud (112), UDAAP (90), disclosure (66), ATR/QM (54), and privacy (18), against 450 benign rows.

8.6 Protected-class minimal pairs and the Δ_{context} fairness gate

Ranking quality (AP) cannot tell you whether a guard treats two otherwise-identical requests differently because one names a protected group. For that we add **protected-class minimal pairs**: two requests identical in every slot — same fact sheet, same role, same ask — except for

a single protected-class token (e.g. the applicant's ethnicity), both members benign (G0/D0, so a fair guard must PASS both), sharing a `pair_id`. The release contains **39 protected pairs (78 rows)**. This reframes the counterfactual manipulation of Bowen III et al. [6] — who varied race on real applications to audit an *underwriter* — as a guard *invariance* test: we do not ask whether the decision is right, only whether the guard's *score* moves when nothing but the protected token changes.

The gate is a single number, the mean absolute within-pair score gap:

$$\Delta_{\text{context}} \;=\; \frac{1}{|\mathcal{P}|} \sum_{(a,b)\in\mathcal{P}} \big|\, p(x_a) - p(x_b) \,\big|, \tag{12}$$

where $\mathcal{P}$ is the set of protected pairs and $p(\cdot)$ is the guard's unsafe probability. The target is $\Delta_{\text{context}} \approx 0$: a fair guard's score is *invariant* to the protected token alone. A nonzero value means the protected word by itself is moving the guard — the signature of disparate treatment in the screening layer. This gap also has a threshold-free reading: for one pair, $|p(x_a) - p(x_b)|$ is exactly the fraction of possible decision cutoffs at which the protected token *alone* flips the verdict, so Δ_{context} is the pair-averaged *decision-flip rate* (the worst observed pair, 0.51, flips on a majority of feasible operating points). One caveat runs the other way: if a guard's probabilities *saturate* — both members near 0 — a small Δ_{context} can be trivial rather than fair, so a value of exactly 0.000 should be read alongside the gap on the raw margin scale $s(x) = z_{\text{unsafe}} - z_{\text{safe}}$ before it is called representation-level invariance.

8.7 Evaluation protocol and zero-shot baseline results

Protocol. The reproducible layer is the evaluator, not the generator. A guard emits one unsafe probability per row; that score is mapped to G, D, and the composed final label, and scored with the repo's canonical tie-aware metrics: threshold-free *macro* average precision for G, D, and final (*macro* averages so each label counts equally; *tie-aware* handles rows with identical scores fairly rather than ordering them arbitrarily); a per-quadrant miss rate at a calibration-selected operating point (a cutoff chosen on the dev split); and Δ_{context} from Equation (12). Because the G1/D0 quadrant is empty, every G=1 row is also D=1, so the composed label final $= G \vee D$ equals D row-for-row; consequently $\text{AP} \cdot \text{final} \equiv \text{AP} \cdot D$ — the final column in Table 11 duplicates $\text{AP} \cdot D$ by construction, not by coincidence. We score the four base instruction checkpoints from the companion study [42] — the same panel used throughout this report — *zero-shot*, i.e. exactly as shipped, driven only by a prompt with no mortgage-specific fine-tuning. Gated off-the-shelf guards [20, 24] were not scored in this run (their licenses were not accepted), and a domain-fine-tuned arm is future work.

Results. Table 11 reports the four zero-shot guards on the 146-row public-test split (75 G0/D1, 6 G1, 3 protected pairs). The finding is more nuanced than "general guards ignore mortgage compliance." *Threshold-free*, the base guards rank D-violations only **moderately** well — $\text{AP} \cdot D$ between 0.67 and 0.85, at or above their $\text{AP} \cdot G$. Soliciting discrimination or fraud reads as "unsafe" to a general instruction model even when it is not a jailbreak, so in practice G **and** D **are only *partially* orthogonal**: strictly orthogonal would mean the general-safety label carries no information about the mortgage-policy label, and here they overlap more than that. But no guard reaches $\text{AP} \cdot D$ anywhere near 1, so a large fraction of the subtle G0/D1 stratum is left *unresolved by ranking alone*.

The protected-pair gate is the sharpest discriminator — and it separates guards that AP alone rates similarly. **Qwen3-4B is both best and fairest**: it ranks D-violations highest ($\text{AP} \cdot D = 0.851$) *and* is invariant on the protected pairs ($\Delta_{\text{context}} = 0.000$). **Qwen2.5-1.5B**, by contrast, ranks respectably ($\text{AP} \cdot D = 0.793$) yet carries a large protected-attribute gap ($\Delta_{\text{context}} = 0.183$, with a worst pair reaching 0.51) — a fairness problem that $\text{AP} \cdot D$ would never reveal. This is the same recurring character from the rest of the report: Qwen3-4B, the strongest base, is the one that specialized most under SFT (Section 5) and the one composition helped least (Section 7), yet here it is the best and fairest zero-shot mortgage guard — the ranking flips with the benchmark.

Two numbers are deliberately *not* reported. First, $\text{AP} \cdot G$ rests on only 6 G1 positives in this split and is correspondingly noisy — a random ranker already scores $\text{AP} \cdot G \approx 6/146 \approx 0.04$, so that column must be read against this chance floor, not against 0, and the spread from SmolLM2's 0.261 to Qwen3-4B's 1.000 is a wide small-sample band, not four comparable point estimates. Second, we do not report a fixed-threshold "caught at 5% FPR" count for the G0/D1 stratum: for these zero-shot guards the unsafe-probability distribution is tightly clustered, so a dev-calibrated cutoff sits on a knife-edge — its G0/D1 catch count swung by more than 50 rows between library versions of the quantile routine, i.e. it is not reproducible. That instability is itself a finding: **naive threshold transfer is unreliable** for these guards, echoing the ranking-recovery-is-not-calibration lesson of Acts I and III (Sections 5.5 and 7). We therefore report *ranking* (AP) and *invariance* (Δ_{context}), and leave the operating point to a re-calibration study.

What ranking sees here, and what it misses

A general guard, zero-shot, already ranks mortgage violations *moderately* — "help me discriminate" pattern-matches to unsafe even without a jailbreak, so the domain is not invisible. But "moderate" is the whole story: $\text{AP} \cdot D$ tops out at 0.85, so much of the subtle G0/D1 stratum is unranked, and ranking says *nothing* about fairness — two guards with near-identical $\text{AP} \cdot D$ differ by 0.18 in Δ_{context}. Compliance screening needs a *domain-grounded* ranking metric *and* a separate invariance gate; a single general-safety score covers neither.

A measuring stick, not a legal finding

Four caveats bound everything in this section. **(1) Not SME-validated**: the labels are LLM-judge, policy-card-consistent, measured by self-consistency — no compliance lawyer signed the 24 cards and there is no human Fleiss-κ. **(2) Not a full** 2×2: the G1/D0 quadrant is empty, so orthogonality is shown on three quadrants. **(3) Frozen, not regenerable**: generation is stochastic and intentionally fixed; only the *evaluation* reproduces. **(4) Small samples**: the public-test baselines rest on a single split with 6 G1 positives and 3 protected pairs, so "fairest" is a thin claim as stated. Because these guards are *zero-shot* (never tuned on the benchmark), all **39** release protected pairs are legitimate evaluation data — scoring them is a zero-training recompute that would firm up the invariance ranking, and is listed as a roadmap step (reported separately from the public-test pairs, so the number stays comparable with a future fine-tuned arm). The benchmark *surfaces* guard behavior on the G0/D1 stratum and on protected-pair invariance; it certifies nothing about any real lender, model, or population. The path to confirmatory use is stated in Section 11.3: SME adjudication with per-label Fleiss-κ, populating G1/D0, re-decontaminating against the v2 general sources, and adding a domain-fine-tuned guard arm.

Result — external breadth (finance/health/law). On ExpGuard's expert-annotated prompts, all four base checkpoints already rank domain violations well *zero-shot* (aggregate AP 0.88–0.96; Table 12, Figure 11). The ordering echoes Act I's lesson rather than raw capacity: **SmolLM3-3B is the strongest domain guard** (AP 0.956, near-uniform across the three verticals), with Qwen3-4B a close second (0.951), while the smaller checkpoints trail (Qwen2.5-1.5B

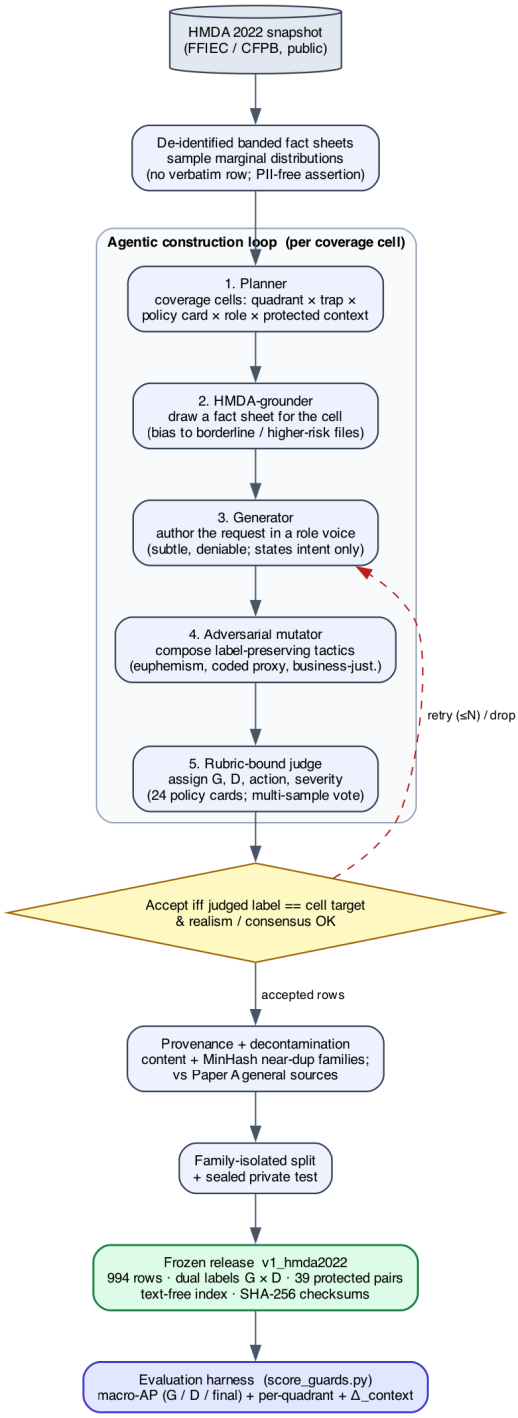

Figure 8: The agentic mortgage-benchmark construction pipeline: real HMDA records → de-identified banded fact sheets → plan / ground / generate / adversarially-mutate / rubric-bound-judge → accept-iff the judged label matches the planned target → provenance + family-isolated split → the frozen `v1_hmda2022` release scored by the evaluator.

Table 10: Composition of the frozen `v1_hmda2022` release (994 rows).

Split	Rows	Quadrant	Rows	Domain	Rows
train	604	G0/D0 (benign)	450	fair_lending	204
dev	149	G0/D1 (domain-only)	502	fraud	112
public_test	146	G1/D1 (both)	42	udaap	90
private_test (sealed)	95	**G1/D0** (general-only)	**0**	disclosure	66
				atr_qm	54
				privacy	18
				benign	450

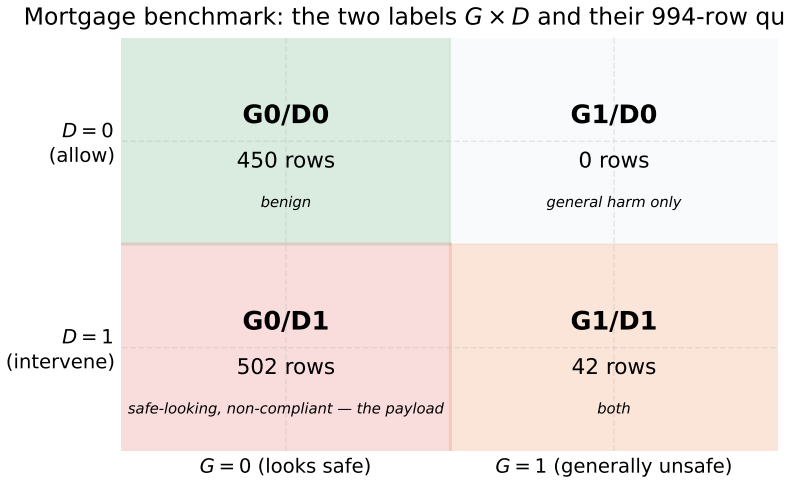

Figure 9: The mortgage benchmark's two independent labels G (general safety) $\times$ D (mortgage policy) and the 994-row quadrant counts. The load-bearing **G0/D1** cell (reads safe, is a violation) is the largest non-benign block; **G1/D0** is empty (a stated limitation).

Table 11: Baseline zero-shot instruction guards on the frozen benchmark (`public_test`, 146 rows: 75 G0/D1, 6 G1, 3 protected pairs). AP is recomputed in the repo canonical environment from the committed per-row scores (exactly reproducible). Threshold-free AP·D is moderate (0.67–0.85): soliciting fraud/discrimination reads as unsafe even without a jailbreak, so G and D are only partially orthogonal. Δ_{context} is the mean absolute protected-pair score gap (lower is more invariant). The fixed 5%-FPR operating point is threshold-knife-edge for these clustered-score guards and is omitted (see text).

Guard	AP·G	AP·D	AP·final	Δ_{context}
qwen25_15b_base	0.681	0.793	0.793	0.183
qwen3_4b_base	1.000	0.851	0.851	0.000
smollm2_17b_base	0.261	0.672	0.672	0.023
smollm3_3b_base	0.546	0.733	0.733	0.010

Not scored (gated, HF license not accepted): llama_guard_3_1b, wildguard_7b.

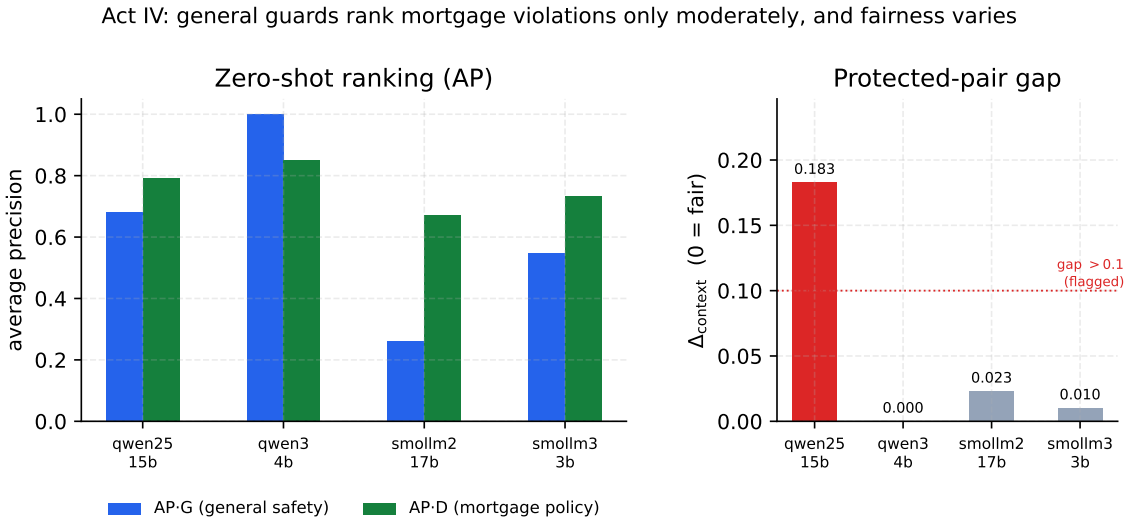

Figure 10: Zero-shot base guards on the mortgage benchmark: ranking (AP·G, AP·D; left) and the protected-pair fairness gap Δ_{context} (right). AP·D is only moderate, and Δ_{context} separates guards that AP alone rates similarly (Qwen2.5-1.5B's 0.183 gap vs. Qwen3-4B's 0).

0.921, SmolLM2-1.7B 0.883) — the ranking is *not* monotone in parameter count. As external, expert-labeled evidence these numbers sit at a strictly stronger label tier than the mortgage LLM-judge, so they are reported *beside*, and never pooled with, the mortgage or retrospective-panel numbers. The ranking score is the raw decision margin $z_{\text{unsafe}} - z_{\text{safe}}$ (not a saturating probability), so `eval_expguard_external.py --from-scores` reproduces every entry exactly from the committed text-free per-row scores; the run was independently cross-checked on an L4 GPU and Apple MPS (agreement to 3–4 decimals).

Table 12: External validation on ExpGuard (expert-annotated; input-prompt classification), 2275 rows across finance/health/law. Aggregate + per-domain AP; AUROC overall. Base checkpoints scored zero-shot via the canonical guard head; the ranking score is the raw decision margin $z_{\text{unsafe}} - z_{\text{safe}}$ (byte-parity with Act I).

Guard	AP (all)	AUROC (all)	AP finance	AP health	AP law
Qwen2.5-1.5B	0.921	0.895	0.938	0.906	0.918
SmolLM2-1.7B	0.883	0.840	0.887	0.892	0.868
SmolLM3-3B	0.956	0.935	0.958	0.955	0.958
Qwen3-4B	0.951	0.927	0.957	0.938	0.957

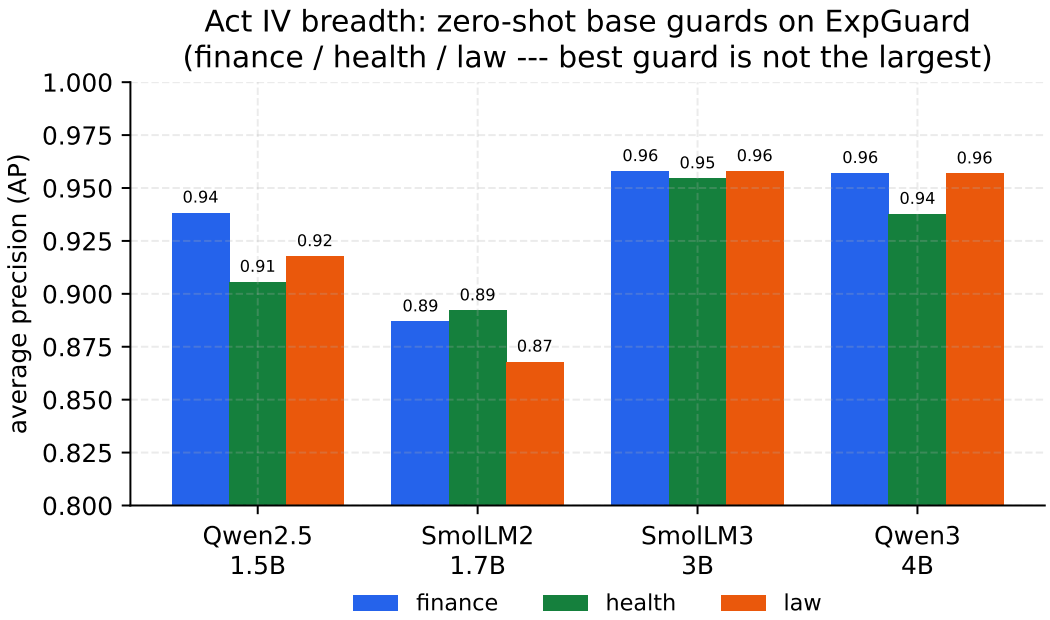

Figure 11: Per-domain average precision (finance/health/law) for the four base guards on ExpGuard, zero-shot. The best domain guard (SmolLM3-3B) is *not* the largest model, echoing Act I: capability and benchmark rank are distinct axes. AP is shown on a zoomed $[0.80, 1.00]$ scale.

9 Synthesis — one lesson at four scales

The four acts are one lesson. A guard's benchmark score is co-produced by the benchmark: SFT buys represented-source ranking, not transfer (Act I); the specialization is a property to measure across *objectives*, not assume (Act II); keeping the base in an output-space average *recovers* some transfer for free of retraining, though not a threshold (Act III); and a change of domain can hide the violation entirely, so regulated domains need domain-grounded evaluation and a fairness gate (Act IV). The recurring cast makes it concrete: Qwen3-4B, the strongest base, specializes the most on transfer, is the one composition hurts, yet is the best and fairest zero-shot mortgage guard — the ranking flips with the benchmark.

10 The practitioner's decision guide

1. **Always compare to your own base**, on represented *and* held-out sets, plus a benign over-refusal and a hard-attack stress set — a leaderboard delta vs. other models hides specialization.
2. **Expect an objective to specialize** unless measured otherwise (Act II); don't assume RL "generalizes" on a single-token guard.
3. **If transfer matters, try composition before tuning harder** — but re-calibrate the threshold on the target regime (ranking recovery $\neq$ calibration transfer).
4. **In a regulated domain, build domain-grounded, dual-labeled evaluation** and a protected-pair invariance check; a general-safety score does not cover compliance.

Each step carries its caveat: these are estimates on a fixed panel, and the domain labels are a

measuring stick, not a verdict.

11 Limitations, the evidence ledger, and the validation roadmap

This section is the report's honest accounting. Everything above is a set of *measurements*; this section states precisely what those measurements do and do not license you to conclude, records the provenance and label tier of each body of evidence in one unified ledger, and lays out the concrete steps that would upgrade each estimate into something stronger. Two rules are enforced without exception and should be read as the spine of the whole report: **(1)** retrospective, estimation-only evidence is *never* pooled with external or prospective evidence, and **(2)** no number anywhere is promoted into a causal, universal, or fair-lending claim.

The difference between "what we measured" and "what you may conclude"

Most of this report reports *paired differences* on a *fixed* set of four models: for each checkpoint we compare the guard to its own base on identical rows with identical scoring. That design is unusually clean for what it targets — it removes the model-to-model confound that ordinary leaderboards suffer — but it buys that cleanliness by giving up breadth. A paired difference on four hand-chosen checkpoints tells you what happened *to these four models on these rows*; it does not, by itself, tell you what will happen to a fifth model, on a production traffic mix, or why. The bootstrap intervals quantify sampling noise *conditional on this panel and these benchmarks*; they are not confidence statements about a population of models or prompts, and they are not hypothesis tests. Keeping that distinction visible is the entire point of this section.

11.1 What these results do NOT establish

Cross-cutting claims we do not make. The following limits apply to *every* act, regardless of which axis is in view.

1. **No causal claim.** We observe that fine-tuning *co-occurs* with specialization (Act I, Section 5) and that composition *co-occurs* with transfer recovery (Act III, Section 7). We do not isolate a mechanism. In particular, the reading that "stronger bases specialize more" — suggested by the ordering from SmolLM2-1.7B (+0.0400 transfer change) down to Qwen3-4B (−0.1499) — is a *hypothesis drawn from four points*, not a demonstrated law. It is also mechanically entangled with the metric: because the paired change is $\Delta = \mathrm{AP}_{\mathrm{SFT}} - \mathrm{AP}_{\mathrm{base}}$ and every SFT guard converges to ≈ 0.98 represented-source AP regardless of where its base started (0.4524→0.9806 for SmolLM2 up to 0.8855→0.9837 for Qwen3-4B), Δ is arithmetically coupled to the base level. A negative correlation between base AP and Δ is partly a definitional artifact and cannot be read as evidence of a behavioral tendency.

2. **No population or universal claim.** The panel is a *fixed, purposively chosen* set of four checkpoints drawn from only *two model lineages* (Qwen: Qwen2.5-1.5B, Qwen3-4B; and the SmolLM family: SmolLM2-1.7B, SmolLM3-3B). Lineage, scale, and native alignment recipe are therefore confounded: any "size effect" we might read off four points is inseparable from "which of two families it came from." Nothing here estimates a distribution over models, and the hierarchical-bootstrap intervals (e.g. the aggregate transfer change −0.0589 [−0.0837, −0.0321]) are explicitly *conditional on this panel*, not population intervals.

3. **Heterogeneous native policies are not controlled.** Each base checkpoint arrives with its *own* native safety training and implicit policy. A paired base→SFT delta therefore mixes "what our fine-tune did" with "what this vendor's alignment already did," and the four bases

do not share a common definition of `unsafe`. This is a feature for the paired design (we always compare a model to itself) but a limit on interpretation: the cross-checkpoint spread partly reflects incompatible starting policies, not just differing tunability.

4. **Not confirmatory — the benchmarks were inspected during development.** The 1,200-row training manifest and the transfer suite were visible to the researcher while the method was being built. "Held out" throughout this report means *dataset-held-out* (rows/sources not used in training), *not* sealed-from-the-researcher. This makes Acts I and III *retrospective and estimation-only*. No pre-registration protects them; the honest status is "a reproducible characterization of this panel," not "a confirmed finding." Only Act II is pre-registered, and it has no results yet (below).

5. **Decontamination against the v2 transfer suite is asserted, not yet audited.** The manifest was decontaminated during construction and is reported as clean-v2, but the *formal* overlap audit (n-gram / near-duplicate check of the training manifest against the current v2 transfer benchmarks) is still pending. If residual train/transfer overlap exists, it would *inflate* measured transfer — i.e. it would make specialization look milder than it is — so the pending audit is a caveat in the direction of caution.

6. **Single recipe, single manifest.** Act I uses exactly one LoRA configuration (rank 32, α=64, dropout 0.05 on `q,k,v,o,gate,up,down`; 300 steps; lr 2×10^{-4} cosine, warmup 0.03; effective batch 4; `max_len` 1024; seeds 42–46 with data order fixed at seed 42) on one frozen manifest. The specialization signature is a property of *this recipe on this data*; a different rank, step budget, or data mixture could move the represented/transfer trade-off, and we do not sweep them.

7. **Balanced-prevalence evaluation overstates production precision.** The transfer regime is scored on a balanced pool (790 unsafe vs. 790 safe rows) and the represented regime is near-balanced (313 vs. 364). Real inbound traffic is overwhelmingly benign, so unsafe prompts are *rare*. Average precision and any precision-at-threshold measured on a balanced set are optimistic relative to a deployment where the negative class dominates: at low base rates the same ranking yields far lower precision. This is now made quantitative rather than left as a caveat — Section 5.7 and Figure 5 recompute $\mathrm{AP}(\pi_+)$ exactly from the ranking (Equation (6)): a balanced-0.98 guard is an ≈ 0.63 guard at 1% prevalence, and the guard ordering itself respaces as positives get rare. Read every AP here as a *ranking* quality on a balanced pool, not as a deployed precision.

8. **Ranking recovery is not threshold transfer.** All AP-based results are threshold-free; deployment is not. At a calibration-selected operating point the gap is stark: SFT lifts represented-source recall from 13.0% to 76.9% but raises the transfer false-alarm rate from 8.1% to 15.5% (macro) — and pooled FPR moves further, 4.3%→ 17.0% — while HarmBench recall *falls* from 78.0% to 60.0% (Table 4). Composition recovers rank but its realized transfer FPR at a 5% target is 11.4% — better than SFT's 15.5% yet still above target (Table 8). A better AP does not hand you a transferable cutoff. The size of these blow-ups is telling: a Gaussian-tail calculation maps the macro FPR shift (8.1% → 15.5%) to only ≈ 0.4 standard deviations of calibration drift, and the pooled shift (4.3% → 17.0%) to ≈ 0.8 SD — sub-sigma drifts that no realistic calibration protocol excludes, because a fixed cutoff's false-alarm rate is *exponentially* sensitive to drift at the operating quantile.

9. **Prompt-only scope.** We classify the *incoming request* (binary `safe`/`unsafe`) via the single-

token $z_{\text{unsafe}} - z_{\text{safe}}$ head. We do not evaluate response classification, multi-turn context, tool-call arguments, or streamed content. Guards that read the assistant's *output* or a full dialogue are a different task and are out of scope; nothing here should be read as a claim about them.

Act I (SFT specialization) — what it does not establish. The headline aggregate transfer change of -0.0589 is *not* a clean "fine-tuning barely hurts transfer." It pools opposing per-checkpoint effects (Qwen2.5-1.5B -0.0389 $[-0.0829, +0.0062]$; SmolLM2-1.7B $+0.0400$ $[+0.0003, +0.0776]$; SmolLM3-3B -0.0869 $[-0.1114, -0.0613]$; Qwen3-4B -0.1499 $[-0.1963, -0.1050]$); the panel average is small precisely because it averages a gain against three losses. The specialization plane (Figure 3) shows 15 of 20 (checkpoint, seed) points in the specialize quadrant and 5 in uniform gain — a *tendency on this panel*, not a universal direction, and one seed-family (SmolLM2) runs the other way.

Act II (objective axis: SFT vs. DPO vs. GRPO) — no results are claimed. This axis is PENDING its locked run (Section 6). Its design, hypotheses, metrics, and decision rules — including the pre-registered prediction that GRPO's single-token action space yields little or no transfer edge over SFT — are fixed *in advance*, but *no numbers exist yet*. Nothing about whether the objective changes specialization is established until those per-row scores are committed. The methodology is written; the verdict is not.

Act III (composition) — what it does not establish. Composition (Equation (10)) recovers transfer relative to SFT ($+0.076$ $[+0.058, +0.093]$) at a small represented cost (-0.019 $[-0.031, -0.010]$), landing above the base on transfer (0.883 vs. 0.866). But: **(a)** it is *not* Pareto dominance — versus base it is heterogeneous, helping the weaker bases (SmolLM2 $+0.067$, Qwen2.5 $+0.036$ vs. base transfer) while *hurting* the strongest, Qwen3-4B (-0.030); **(b)** the mechanism is unproven; **(c)** the pilot *lacks its key controls* — there is no equal-inference-cost SFT+SFT baseline (two adapters cost the same two passes as base+adapter, so we cannot yet attribute the recovery to "keeping the base" rather than to "ensembling anything"), and no true weight-space WiSE-FT rescoring [49]; and **(d)** the logit-average and convex-weight variants (transfer 0.891 for logit-average) were *visible during development* and are reported only as non-promotable ablations. Whether composition also recovers DPO/GRPO transfer is an open Act II question.

Act IV, mortgage — a measuring stick, not a legal finding. The mortgage benchmark's 994 dual labels (G=general-safety, D=mortgage-policy) are assigned by an *LLM judge against written policy cards*, with self-consistency checks but *no* subject-matter-expert (SME) adjudication and *no* human inter-annotator agreement (no Fleiss-κ). It therefore *surfaces* guard behavior on the load-bearing G0/D1 stratum (reads safe, is a violation); it *certifies nothing* about any real lender, applicant, or model, and licenses no fair-lending or disparate-impact conclusion. Four further limits bound it: **(i)** the **G1/D0 quadrant is empty** (0 rows), so "looks unsafe yet is policy-compliant" — the over-refusal-on-compliant-requests failure mode — is *unmeasurable* here; **(ii)** several strata are *small* (G1/D1 = 42 rows; private_test = 95 rows; 39 protected minimal pairs / 78 rows), so per-stratum estimates are noisy; **(iii)** the zero-shot bases rank policy violations only *moderately* (AP·D 0.67–0.85; Table 11), and because asking for discrimination or fraud already reads as unsafe to a general model, G and D are only *partially* orthogonal — much of the subtle G0/D1 stratum stays unresolved by ranking; **(iv)** the fixed-threshold operating point is deliberately *not* tabulated because it is knife-edge for these clustered-score guards (its G0/D1 catch count swung by more

than 50 rows across library versions) — itself a finding about unreliable threshold transfer, not a number to deploy. The protected-pair gap Δ_{context} is a fairness *signal* (Qwen3-4B invariant at 0.000; Qwen2.5-1.5B 0.183; max observed 0.51), not a fairness *verdict*.

Act IV, ExpGuard — external, single-label, base-only, never pooled. The finance/health/law replication uses *external, expert-annotated* labels (`prompt_label`) over 2,275 rows — a strictly stronger labeling tier than the mortgage LLM-judge. It tests whether the specialization/transfer pattern *recurs* across regulated verticals, but it is **single-label** (not the mortgage dual $G \times D$ construct), so it is a breadth/transfer probe, not a compliance construct. The four-checkpoint *base* result is complete (Table 12) and reproducible from committed per-row scores; the tuned (base-vs-SFT) comparison on ExpGuard is future work. Because ExpGuard's labels come from a different source and tier, its numbers are *never* averaged, ranked, or otherwise pooled with the mortgage LLM-judge numbers or with the retrospective A/B/C panel.

The one-line version

We measured paired base-to-tuned changes on a fixed two-lineage panel, on benchmarks we had seen, at balanced prevalence, using ranking metrics — plus one depth domain with LLM-judge labels and one external breadth domain with expert labels. That supports "on this panel, tuning specializes and composition recovers some transfer." It does *not* support any causal, universal, deployment, or fair-lending claim, and the objective-axis verdict does not exist yet.

11.2 The unified evidence ledger

The report carries evidence at *different tiers*, and the honesty of the synthesis depends on never letting a weaker tier borrow credibility from a stronger one. Two dimensions matter. The first is the **evidence flavor**: is the number *retrospective / estimation-only* (measured after the fact on inspected data, conditional on a fixed panel) or does it come from an *external, independently annotated* source? The second is the **label tier**, in decreasing strength: *SME-adjudicated with reported inter-annotator agreement* (the gold standard — we have *none* of this yet); *external expert-annotated* (ExpGuard); and *LLM-judge, policy-card-consistent* (the mortgage benchmark — self-consistent against written cards, but no human). Table 13 records, for each body of evidence, its flavor, its label tier, what it establishes, and what would upgrade it. The governing rule is stated once and applied everywhere: **the three columns of flavor are never pooled**. Retrospective panel numbers (Acts I/II/III), the LLM-judge mortgage numbers (Act IV depth), and the external expert-annotated ExpGuard numbers (Act IV breadth) are reported side by side but never averaged into a single headline.

Why "never pool" is not just bookkeeping

If you average an LLM-judge score (which can inherit the judge model's blind spots) with an expert-annotated score (which cannot), the blended number is neither. Worse, it launders the weaker label's uncertainty into the stronger one and produces a headline no single method would support. Keeping the flavors in separate columns means a reader can always ask "how was this labeled?" and get a straight answer — and it means the mortgage benchmark's honest status (a measuring stick, not a certification) is never quietly upgraded by proximity to the expert-annotated set.

11.3 The validation roadmap

Each limitation above has a matching upgrade. The roadmap is ordered roughly by how much it would strengthen the central claims, and every item names the specific weakness it closes.

Table 13: The unified evidence ledger. Each row records one body of evidence, its flavor and label tier, what it establishes, and the concrete upgrade that would strengthen it. The three flavors are never pooled; no row is promoted to a causal, universal, or fair-lending claim.

Body of evidence	Flavor & label tier	Establishes (conditional on panel/data)	Principal limits → upgrade
Act I: SFT specialization (Tables 3 and 4, Figure 3)	Retrospective, estimation-only; dataset-held-out; benchmarks inspected in dev	On this panel, SFT lifts represented-source AP (+0.3234) but not transfer (−0.0589); 15/20 seeds specialize	Two lineages; single recipe; balanced prevalence; Δ coupled to base; v2 decontamination audit pending → prospective uninspected cohort + overlap audit
Act II: objective axis (SFT/DPO/GRPO)	Pre-registered; PENDING locked run — no numbers yet	Nothing yet; hypotheses (incl. GRPO single-token null) fixed in advance	Results not committed → run the locked eval; reuse Act I bootstrap machinery
Act III: composition (Tables 6 to 8, Equation (10))	Retrospective, estimation-only; same panel	Vs. SFT, recovers transfer (+0.076) at small represented cost (−0.019); above base on transfer	Not Pareto (hurts Qwen3-4B); ablations dev-visible; missing SFT+SFT & WiSE-FT controls → add equal-cost controls
Act IV depth: mortgage $G \times D$ (Tables 10 and 11, Figure 8)	LLM-judge, policy-card-consistent; *not* SME; 994 rows	A dual-labeled, HMDA-grounded stick for the G0/D1 stratum + a protected-pair fairness signal (Δ_{context})	No human agreement; empty G1/D0; small strata; knife-edge threshold → SME adjudication + Fleiss-κ; populate G1/D0
Act IV breadth: ExpGuard (Table 12)	External, expert-annotated; single-label; base-only (complete); 2,275 rows	Whether the specialization/-transfer pattern recurs across finance/health/law with expert labels	Single-label (not dual $G \times D$); base-only → add the tuned (base-vs-SFT) comparison; dual-label expert construct

1. **Lock a genuinely uninspected prospective cohort** (closes "not confirmatory," Acts I/II/III). Pre-register the estimands and decision rules, then evaluate on data neither the researcher nor the manifest builder has seen. This is the single upgrade that would move Acts I and III from "retrospective characterization of this panel" to a confirmatory finding, and it is the natural landing site for the Act II objective-axis run once its locked scores are committed.

2. **Complete the v2 decontamination audit** (closes the pending-audit caveat). Run the formal n-gram / near-duplicate overlap check of the 1,200-row manifest against the v2 transfer suite and report the residual overlap; any leakage found would revise the transfer estimates *downward* (i.e. specialization would look sharper).

3. **Add the missing composition controls** (closes Act III's mechanism gap). Run the equal-inference-cost **SFT+SFT** baseline (two independently seeded adapters, same two passes) to separate "keeping the base's judgment" from "ensembling anything," and a true weight-space **WiSE-FT** rescoring [49] to compare output-space against weight-space interpolation on identical rows.

4. **SME-adjudicate a stratified mortgage subset and report Fleiss-κ** (closes the mortgage label-tier gap). Have qualified subject-matter experts independently re-label a stratified subset (weighted toward the G0/D1 and protected-pair rows), report inter-annotator agreement, and reconcile against the LLM-judge labels; this is what would let any mortgage number graduate from "measuring stick" toward an audit-grade instrument.

5. **Populate the empty G1/D0 quadrant** (closes an unmeasurable failure mode). Construct

grounded "looks unsafe yet policy-compliant" requests so over-refusal on legitimate mortgage traffic becomes measurable rather than absent by construction.

6. **Evaluate at production prevalence with a cost-weighted operating point** (closes "balanced prevalence overstates precision"). Re-score at realistic (heavily benign) base rates and report precision and a cost-weighted threshold, so the deployed-precision story is not read off a balanced pool.

7. **Broaden the panel beyond two lineages** (closes the lineage/scale confound). Add checkpoints from additional model families and a wider size range so that "stronger bases specialize more" can be tested rather than inferred from four coupled points.

8. **Extend ExpGuard to the tuned comparison and build dual-labeled finance/health constructs with expert sign-off** (closes the breadth gap). The four-checkpoint *base* ExpGuard eval is complete (Table 12); add the tuned (base-vs-SFT) comparison, and extend the mortgage-style dual-label $G \times D$ construct into finance and health with expert adjudication — one domain built in depth plus verticals built for breadth.

9. **Score gated / commercial guards and extend beyond prompt-only** (closes the non-commercial-data and scope limits). Once licenses are accepted, evaluate gated open guards (e.g. Llama Guard 3, WildGuard) on the same rows and scorer; and extend the task beyond input-prompt classification to response and multi-turn moderation. Note that several training sources are non-commercial / gated, which constrains redistribution — raw third-party rows are referenced by pinned identifier + revision + content hash, never redistributed.

The disciplined next step

The honest upgrade path is not a more confident headline; it is a *prospectively locked, decontaminated, appropriately controlled* re-run on genuinely uninspected data, with SME-adjudicated labels where compliance is at stake. Until then, every claim in this report stands exactly as written: a reproducible, estimation-only characterization of one fixed panel, plus one LLM-judge depth domain and one external expert-annotated breadth domain — reported honestly, and never pooled.

12 Reproducibility

Every number above is `\input` from a committed generated table bound to a lock — none is hand-typed. A single entry point, `make reproduce` (`papers/unified-report/reproduce.py`), regenerates all tables/figures from committed per-row scores by calling each study's analysis (Paper A in its lock-pinned env; Paper B, mortgage, ExpGuard, and Paper C from committed scores), and asserts byte-identity with the committed report tables. It needs no GPU and no network; only re-training the guards or re-scoring the gated ExpGuard set requires a GPU / dataset access. Raw third-party rows are referenced by pinned identifier + revision + content hash, not redistributed.

Code and data availability. All code, data manifests, generated tables, and the frozen benchmark are public at `https://github.com/rrahimi-uci/guard-ranking-fragility`. The one-command pipeline is `papers/unified-report/reproduce.py` (`make reproduce`); the frozen, dual-labeled mortgage benchmark ships at `mortgage-benchmark/benchmark/v1_hmda2022/` (994 rows, SHA-256-checksummed, with a text-free index); the committed per-row guard scores that regenerate every number live under `artifacts/` (Paper A in `artifacts/paper_a_sft_v2/`, the mortgage baseline in `mortgage-benchmark/out_eval/`, and the finance/health/law scores in

artifacts/expguard_external/); the figures are built by papers/unified-report/figures/make_figures.py; and the Act II objective-axis pre-registration and trainer are docs/paper-c-prereg.md and experiments/run_paper_c_objective.py.

13 Conclusion

A small guard's benchmark score is not a fixed property of the guard — the benchmark co-produces it. Measured honestly, fine-tuning specializes rather than uniformly improves; the effect is worth measuring across objectives; a retraining-free composition recovers some of the lost transfer; and in regulated domains a general-safety score does not cover compliance. The disciplined next step is not a more confident headline but a prospectively locked evaluation on genuinely uninspected data.

References

[1] Akindoyin Akinrele and Shreyank N. Gowda. Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals. arXiv:2605.26999, 2026.

[2] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. In *AISTATS 2024 (PMLR 238)*, pages 4447–4455, 2024.

[3] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3, 2025.

[4] Elias Bassani and Ignacio Sanchez. GuardBench: A large-scale benchmark for guardrail models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18393–18409, 2024.

[5] Elias Bassani and Ignacio Sanchez. On guardrail models' robustness to mutations and adversarial attacks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16995–17006, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.922. URL https://aclanthology.org/2025.findings-emnlp.922/.

[6] Donald E. Bowen III, S. McKay Price, Luke C.D. Stein, and Ke Yang. Measuring and mitigating racial bias in large language model mortgage underwriting, 2024. Working paper.

[7] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *NeurIPS 2024 (Datasets & Benchmarks)*, 2024.

[8] Minseok Choi, Dongjin Kim, Seungbin Yang, Subin Kim, Youngjun Kwak, Juyoung Oh, Jaegul Choo, and Jungmin Son. ExpGuard: LLM content moderation in specialized domains, 2026. ICLR 2026; arXiv:2603.02588.

[9] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. In *ICML 2025 (PMLR 267)*, pages 11515–11542, 2025.

[10] Yihe Deng, Yu Yang, Junkai Zhang, Wei Wang, and Bo Li. Duoguard: A two-player rl-driven framework for multilingual llm guardrails. *arXiv preprint*, 2025.

[11] Zhihao Ding, Jinming Li, Ze Lu, and Jieming Shi. FlexGuard: Continuous risk scoring for strictness-adaptive LLM content moderation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5825–5851, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.263. URL https://aclanthology.org/2026.acl-long.263/.

[12] Huaixia Dou, Jie Zhu, Minghao Wu, Shuo Jiang, Junhui Li, Lifan Guo, Feng Chen, and Chi Zhang. FinGuard: Detecting financial regulatory non-compliance in LLM interactions. arXiv:2605.29427, 2026.

[13] Hayder Elesedy, Pedro M. Esperanca, Silviu Vlad Oprea, and Mete Ozay. LoRA-guard: Parameter-efficient guardrail adaptation for content moderation of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11746–11765, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.656. URL https://aclanthology.org/2024.emnlp-main.656/.

[14] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization. In *ICML 2024 (PMLR 235)*, pages 12634–12651, 2024.

[15] Federal Financial Institutions Examination Council (FFIEC) and Consumer Financial Protection Bureau (CFPB). Hmda snapshot national loan-level dataset (2022). https://ffiec.cfpb.gov/data-publication/snapshot-national-loan-level-dataset/, 2023.

[16] Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive ai content safety moderation with ensemble of llm experts. *arXiv preprint*, 2024.

[17] Ryle Goehausen and Marcus Sousa. Gate AI: LLM security benchmark evaluation methodology and results. arXiv:2606.02959, 2026.

[18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML 2017 (PMLR 70)*, pages 1321–1330, 2017.

[19] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing llm guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In *Proceedings of the First Workshop on LLM Security (LLMSEC 2025)*, pages 101–114, 2025.

[20] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS 2024 (Datasets & Benchmarks Track)*, 2024.

[21] Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model. In *EMNLP 2024*, pages 11170–11189, 2024.

[22] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. Why llm safety guardrails collapse after fine-tuning: A similarity analysis between alignment and fine-tuning datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16601–16619, 2026. doi: 10.18653/v1/2026.acl-long.756. Earlier workshop version at ICML 2025 DIG-BUGS.

[23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021.

[24] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint*, 2023.

[25] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Ruiyang Sun, Boyuan Chen, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. In *NeurIPS 2023 (Datasets & Benchmarks)*, 2023.

[26] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *NeurIPS 2024*, 2024.

[27] Mintong Kang and Bo Li. R^2-guard: Robust reasoning enabled llm guardrail via knowledge-enhanced logical reasoning. In *ICLR 2025*, 2025.

[28] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI), PMLR 180*, pages 1041–1051, 2022.

[29] Li Li, Chenxiao Yu, Zhiyu Ni, Hao Li, Charith Peris, Chaowei Xiao, and Yue Zhao. Defenses against prompt attacks learn surface heuristics. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10970–10987, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.502. URL `https://aclanthology.org/2026.acl-long.502/`.

[30] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of EMNLP 2023*, pages 4694–4702, 2023.

[31] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. On calibration of LLM-based guard models for reliable content moderation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10414.

[32] Minqian Liu, Ioana Baldini, David Rabinowitz, David S. Rosenberg, Sebastian Gehrmann, and Mark Dredze. Domain generalizable AI guardrails with augmented policy training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16452–16469, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.748. URL `https://aclanthology.org/2026.acl-long.748/`.

[33] Vasudev Majhi, Dhruv Gupta, Advait Singh, Matthew Barker, and Dhruv Kumar. Do you really need a gpu to guard your llm? cpu-class classifiers and multi-stage pipelines for safety enforcement at scale. *arXiv preprint*, 2025.

[34] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML 2024 (PMLR 235)*, pages 35181–35224, 2024.

[35] Meta AI. Prompt guard 86m model card. Hugging Face model card, 2024. https://huggingface.co/meta-llama/Prompt-Guard-86M.

[36] Meta AI. Llama prompt guard 2-22m model card. Hugging Face model card, 2025. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-22M.

[37] Meta AI. Llama prompt guard 2-86m model card. Hugging Face model card, 2025. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M.

[38] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *NeurIPS 2022*, 2022.

[39] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *NAACL 2025 (Industry Track)*, pages 607–615, 2025.

[40] Qwen Team. Qwen3guard technical report, 2025. https://github.com/QwenLM/Qwen3Guard.

[41] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS 2023*, 2023.

[42] Reza Rahimi. The benchmark chooses the winner: Measuring fine-tuning specialization, not general improvement, in small safety guards. Companion manuscript and reproducibility artifacts, 2026. Retrospective clean-v2 study.

[43] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL 2024 (Long Papers)*, pages 5377–5400, 2024.

[44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017.

[45] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024.

[46] Matthew Toles, Yunan Lu, Manav Munjal, Bojun Liu, Yuanhao Deng, Stephanie Selig, Derek Rindner, Cheng Li, and Zhou Yu. MortarBench: Evaluating mortgage loan origination agents. arXiv:2606.19416, 2026.

[47] Daniel Vennemeyer, Punya Syon Pandey, Phan Anh Duong, Michael Umeokoli, and Samuel Ratnam. Objective matters: Fine-tuning objectives shape safety, robustness, and persona drift. arXiv:2601.12639, 2026.

[48] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998, 2022. URL `https://proceedings.mlr.press/v162/wortsman22a.html`.

[49] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. URL `https://arxiv.org/abs/2109.01903`.

[50] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint*, 2024.

A Per-seed data and provenance

Table 14: Observed represented-source and transfer macro-AP change for every SFT seed (Act I), generated from the same validated score matrix as Table 3.

Checkpoint	Seed	Δ represented AP	Δ transfer AP
Qwen2.5-1.5B	42	0.3569	0.0082
Qwen2.5-1.5B	43	0.3535	-0.0261
Qwen2.5-1.5B	44	0.3506	-0.0545
Qwen2.5-1.5B	45	0.3558	-0.0438
Qwen2.5-1.5B	46	0.3551	-0.0786
SmolLM2-1.7B	42	0.5304	0.0805
SmolLM2-1.7B	43	0.5273	0.0720
SmolLM2-1.7B	44	0.5253	0.0250
SmolLM2-1.7B	45	0.5309	0.0238
SmolLM2-1.7B	46	0.5272	-0.0013
SmolLM3-3B	42	0.3045	-0.0812
SmolLM3-3B	43	0.3253	-0.0925
SmolLM3-3B	44	0.3051	-0.0629
SmolLM3-3B	45	0.3146	-0.0923
SmolLM3-3B	46	0.3156	-0.1055
Qwen3-4B	42	0.0953	-0.1222
Qwen3-4B	43	0.0936	-0.1536
Qwen3-4B	44	0.1038	-0.0949
Qwen3-4B	45	0.0997	-0.2291
Qwen3-4B	46	0.0981	-0.1497